Lecture 10

NP 完全性

10.1 引言

激动人心的，我们来到了这门课的一个新的转折点。之前的讨论介绍了几种算法设计策略，看到了很多能巧妙利用问题结构从而可以快速解决的问题，介绍了非常多如何“解决”某个问题的方法。现在需要考虑的问题是，一个问题有多难，是否存在一种方法使得它能够被高效地解决？这个问题显然比先前的考虑更加抽象，也需要更多的技巧和思考，事实上也是理论计算机科学的终极目标之一。在通常的意义下认为能“快速解决”或“高效解决”的问题是指存在一个多项式时间的算法能够解决的问题，而所谓多项式时间，是指算法的时间复杂度与输入的长度之间是多项式关系。下面来看三个经典的存在多项式时间算法的问题：

1. 最短路径问题：给定一个没有负环的有向图 $G=(V,E)$，即使带有负权边，也可以用 Bellman–Ford 在 $O(|V||E|)$ 次松弛内求出从单一源点出发的最短路径；
2. 欧拉回路问题：是否存在一个回路（即起点终点是同一个顶点）使得它恰好经过图中每条边一次？这个问题在数据结构基础课程中已经讨论过，可以在 $O(|V|+|E|)$ 的时间内解决；
3. 2-CNF 可满足性问题（简称 2-SAT 问题）：回忆离散数学，称一个逻辑表达式是 k-CNF（即 k-合取范式）的，如果它是若干子句的合取（$\wedge$），每个子句是至多 k 个文字的析取（$\vee$），其中文字是变量或者变量的否定。2-SAT 问题就是判断一个 2-CNF 逻辑表达式是否存在对其变量的某种 0 和 1 的赋值，使得它的值为 1，这个问题可以在多项式时间内解决。例如

   $$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

   满足赋值条件 $x_1=1, x_2=0, x_3=1$。这一问题与图论中的强连通分量有关，在后续介绍了归约的概念后会详细讨论，总之也是多项式时间可解的。

0-1 背包的动态规划却需要特别注意。它的时间为 $O(nC)$，其中 n 是物品个数，C 是背包容量。正整数 C 只需 $\lfloor \log_2 C \rfloor + 1$ 位，因此 $O(nC)$ 并不是关于输入位长的多项式上界。这类关于输入数值有多项式上界的算法称为伪多项式算法。物品数 n 没有同样的问题，因为输入会逐件列出物品，至少需要写出 n 项数据。

那么，0-1 背包问题能不能找到一个多项式时间算法呢？遗憾的是，至今还没有答案。前面关于算法设计的讨论仿佛给了我们很大的信心，很多问题都能找到巧妙的解法，但只要对刚才的三个问题稍作修改，就会发现事情并没有那么简单：

1. 最短路问题变体：如果允许负边，并要求给出不重复顶点的最短路径，那么即使图中有负环，也必须在所有简单路径中找出最短的一条，这个问题目前尚无已知的多项式时间算法；

2. 哈密顿回路问题：起点和终点是一个顶点，途中经过图中所有其他节点且只经过一次。即将欧拉回路每条边经过一次改为了每个点经过一次，然而判断一个图是否存在哈密顿回路目前也没有已知的多项式时间算法，尽管看起来和欧拉回路区别不大；
3. 3-CNF 可满足性问题（简称 3-SAT 问题）：判断一个 3-CNF 逻辑表达式是否存在对其变量的某种 0 和 1 的赋值，使得它的值为 1。这个问题目前也没有多项式时间的算法，尽管和 2-SAT 问题仅有一字之差。

由此可以发现，似乎问题与问题之间亦有差别：有的问题能找到很巧妙的方法在多项式时间内解决，而有的问题虽然看起来和前者差不多，但至今也没有找到多项式时间的算法。这也就带来了这一节讨论的核心：如何形式化地定义问题的困难程度，如何对这些问题按照困难程度进行分类，这些困难程度背后又有什么有趣的内涵？为了讨论这些问题，需要首先形式化地定义问题以及计算这些问题的形式化计算模型。接下来开始正式的讨论。

10.2 形式语言与输入编码

前面的问题涉及图、整数和逻辑公式，直接比较它们似乎无从下手。不过，计算机接收这些对象时，都会先把它们表示成字符串。因此，我们可以先从字符串出发，用同一种方式描述这些问题。

10.2.1 用字符串表示问题

先取一个有限的非空集合 Σ，称为**字母表（alphabet）**。把其中有限个字符依次连接，就得到一个**字符串（string）**。例如，取 $\Sigma = \{0, 1\}$，便得到二进制字符串。没有任何字符的字符串称为空串，记作 ε。包括空串在内，所有有限字符串组成的集合记作 Σ^*。

字符串本身并没有指定它表示什么。例如，下面这个二进制字符串

$$01001000011001010110110001101100011011111$$

可以表示十进制整数 310939249775，也可以每八位一组，按 ASCII 编码读成 Hello。所以，使用字符串表示输入时，首先要约定怎样解释它。

约定了字符的含义，也不代表任意拼出的字符串都符合我们的要求。例如

$$\text{军进犯大林嘴鸟军强脾气穷就怕嗲气}$$

这是一段脸滚键盘打出来的字，它也是中文字符上的字符串，但我们认为它毫无意义。我们可以从所有字符串中挑出满足某种要求的那些字符串，组成集合 L。这样的集合称为**形式语言（formal language）**，也就是 Σ^* 的一个子集。

现在来看怎样用语言描述前面的 Hamilton 回路问题。假设已经约定了图的编码方式，用 $\langle G \rangle$ 表示图 G 的编码。将所有具有 Hamilton 回路的图的编码放在一起，就得到语言

$$\text{HC} = \{\langle G \rangle : G \text{ 是有 Hamilton 回路的有限简单无向图}\}.$$

给定一个图，判断它有没有 Hamilton 回路，就变成了判断它的编码是否属于 HC。这种只要求回答“是”或“否”的问题称为**判定问题（decision problem）**。一般地，一个判定问题对应的语言，就是所有肯定实例的编码组成的集合。

如果题目要求找出一条 Hamilton 回路，就还要给出具体的顶点访问顺序，这称为**搜索问题**（**search problem**）。我们可以用关系 $R(x, y)$ 表示“y 是输入 x 的一个合法解”，要求算法找到满足这个关系的 y，或者报告不存在。对于 Hamilton 回路，x 表示图，y 表示顶点排列，检查的条件是排列中相邻顶点以及首尾顶点之间都有边。

再给每条边加上长度，要求找总长度最小的 Hamilton 回路，就得到旅行商问题这样的**优化问题**（**optimization problem**）。它也有对应的判定问题，即给定一个数 K，问是否存在长度不超过 K 的回路。求出最优值以后，和 K 比较就能回答这个问题。至于能否反过来利用判定算法找出具体的解，我们将在第 10.4.4 节讨论。

语言本身是集合，因此可以像普通集合一样取并、交与补。例如，$L_1 \cap L_2$ 包含同时属于两种语言的字符串，而补语言 $\overline{L} = \Sigma^* \setminus L$ 包含字母表上所有不属于 L 的字符串。字符串还可以首尾相接，由此得到语言的连接运算。从 L_1 中取一个字符串 x，从 L_2 中取一个字符串 y，把 y 接在 x 后面，所得的所有字符串组成 $L_1L_2 = \{xy : x \in L_1,\ y \in L_2\}$。如果反复从同一个语言 L 中取字符串并连接，可以定义 L^k 为恰好连接 k 个字符串所得的集合。允许连接任意有限个字符串，就得到 L 的 Kleene 星 $L^* = \bigcup_{k=0}^{\infty} L^k$，其中 $L^0 = \{\varepsilon\}$。这里取零个字符串时得到空串，所以空串总在 L^* 中。例如，$L = \{01\}$ 时，$L^* = \{\varepsilon, 01, 0101, \ldots\}$。注意 $\{\varepsilon\}$ 包含一个字符串，而空语言 $\varnothing$ 一个字符串也不包含。

10.2.2 编码与输入长度

上一节把图表示成字符串，再用语言描述 Hamilton 回路问题。接下来讨论时间复杂度时，还需要知道这个字符串有多长。同一个对象怎样写成输入，会影响我们对运行时间的判断。

先看一个整数。例如，整数 8 用二进制写成 1000，只需四位，若用连续的 1 的个数表示它，就要写成 11111111，需要八位。对于一般的正整数 C，这两种写法分别需要 $\lfloor \log_2 C \rfloor + 1$ 位和 C 位。这也解释了引言中背包算法的问题。动态规划的时间为 $O(nC)$，其中 n 是物品数，C 是容量。如果容量真的用 C 个字符输入，那么逐个处理这 C 种容量并不超过输入长度的多项式。但通常输入的是 C 的二进制表示，长度约 $\log_2 C$ 位，因此 $O(nC)$ 不能保证算法在输入长度的多项式时间内完成。

图也有不同的表示方式，不过常用的邻接矩阵和邻接表之间没有这么大的差距。对于顶点编号为 $1, \ldots, n$ 的简单无向图，可以先记录顶点数，再逐行写出邻接矩阵。为了标出顶点数在哪里结束，一种简单的办法是先写 n 个 1，再写一个 0 作为结束标记，随后写出矩阵的 n^2 个元素。例如，只有两个顶点和连接它们的一条边时，矩阵为 $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$，对应的字符串可以写成 110 0110，其中空格仅用于分开顶点数与矩阵，实际不必存储。读入时数出开头的两个 1，就知道接下来应按一个 2×2 的矩阵读取。

若改用邻接表，则逐个顶点列出它的邻居。已有邻接矩阵时，扫描所有矩阵元素就能生成邻接表。反过来，已有邻接表时，先建立全零矩阵，再按表中的边将相应位置改成 1 即可。因此这两个转换都只需要输入长度的多项式时间。因此，如果某个图问题已有使用邻接矩阵的多项式时间算法，收到邻接表后，先转换再运行它，仍然是多项式时间，反过来也一样。

一般地，设原输入长为 N，转换为另一种表示需要 $O(N^r)$ 时间。因为输出字符也要花时间，转换后长度至多为 $O(N^r)$。假设求解算法对长度为 M 的输入需要 $O(M^s)$ 时间，那么把转换后的输入交给它，求解时间至多为 $O(N^{rs})$，连同转换共需 $O(N^r + N^{rs})$ 时间，仍是 N 的多项式。这就是为什么两种表示能够在多项式时间内相互转换时，选择哪一种都不会改变问题的多项式时间可解性。

10.3 计算模型与 P、NP

10.3.1 图灵机

上一节把问题写成了对字符串的判定，现在还需要说明计算机怎样完成这个判定。考虑计算过程，最自然的想法就是算法：按照事先规定的指令，一步一步地处理输入，最终得到答案。此前我们一直在寻找这样的算法，但还可以进一步追问，是否有些问题无论怎样设计算法都无法解决？要严格回答这个问题，就需要先把算法能够执行的操作明确下来。图灵提出的计算模型，正是为了用简单而明确的规则描述这样的计算过程。

定义 10.1. 图灵机有一条向左右无限延伸的纸带（tape），划分成一个个格子（cell），每格存放一个符号。这些符号取自有限的纸带字母表 Γ，其中包括空白符 $\square$。输入使用的符号组成输入字母表 Σ，满足 $\Sigma \subseteq \Gamma \setminus \{\square\}$。

一个读写磁头（head）指向纸带上的某个格子，图灵机还保存着当前状态，取自有限状态集 Q。其中指定 q_0 为起始状态，q_h 为停机状态。每一步，图灵机根据当前状态和磁头读到的符号，完成以下操作：

1. 将当前格子的符号改写为指定符号；
2. 转入指定状态；
3. 将磁头向左或向右移动一格，或者留在原地。

这些操作由转移函数

$$\delta : (Q \setminus \{q_h\}) \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$$

决定，其中 L, R, S 分别表示左移、右移和留在原地。例如，$\delta(q, a) = (q', b, R)$ 表示图灵机在状态 q 下读到 a 时，将当前格子改写为 b，转入状态 q'，并将磁头右移一格。当前状态和所读符号确定后，下一步的操作便唯一确定，因此称为**确定性图灵机（deterministic Turing machine）**。开始时，将输入 x 依次写在从第 0 格起的 $|x|$ 个格子中，其余格子为空白，磁头指向第 0 格，图灵机处于 q_0。此后按照转移规则逐步执行，进入 q_h 时停止，并按约定读取纸带上的计算结果。

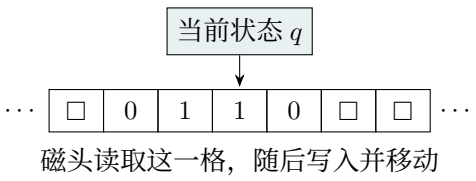


图 10.1 一个图灵机构型的局部：当前状态、纸带内容与磁头位置共同决定下一步。

要知道图灵机下一步会怎样行动，需要知道它当前处于什么状态，磁头在哪里，以及纸带上写着什么。这些信息合起来称为图灵机的**构型（configuration）**。转移函数确定后，每个构型都唯一决定下一构型。下面我们来看两个基本的图灵机的例子。

例题 10.2. 用 1^x 表示整数 x，要求输出 1^{x+2}，也就是计算 $x + 2$。自然地，图灵机应当先找到输入末尾，再写入两个 1。下面的三个转移就能完成这一过程。

当前状态	读到	新状态	写入	方向
q_1	1	q_1	1	R
q_1	□	q_2	1	R
q_2	□	q_h	1	R

图灵机从 q_1 开始，读到 1 就继续向右走，直到遇到原输入后面的空白格。此时写入第一个 1，进入 q_2，再向右写入第二个 1，便完成了加二并停机。

例题 10.3. 在判定问题中，图灵机最终需要回答“是”或“否”，可以在纸带上留下 1 或 0 表示答案。也可以分别设置接受状态 q_{acc} 和拒绝状态 q_{rej}，进入相应状态就表示计算结束，并给出对应答案。若在这两个状态后分别增加写入 1 或 0、再进入 q_h 的操作，就回到了定义中只有一个停机状态的写法。后面讨论判定问题时，采用接受和拒绝这两个状态。

如果图灵机对每个 $x \in L$ 都接受，对每个 $x \notin L$ 都拒绝，称它**判定（decide）** L，也就是说，它必须在每个输入上停机并给出答案。如果只要求“接受当且仅当 $x \in L$”，允许在非成员上一直运行，称它**识别（recognize）** L。能识别不一定能判定，补充材料中讨论的停机问题就是一个例子。

现在图灵机已经给出了一个明确的计算模型，但它能否表达我们通常所说的算法？Church–Turing 论题将这两者联系起来。

论题 10.4 (Church–Turing 论题)**.** 任何能够按照有限条明确的规则逐步计算的函数，都可以由图灵机计算。

虽然图灵机已经有严格的数学定义，但“按照有限条明确的规则逐步计算”仍是直观描述，尚未给出严格的数学定义。Church–Turing 论题认为，图灵机恰好能够表达这种计算能力。由于其中一方仍是直观概念，这一主张不能像两个数学模型之间的等价关系那样直接证明，因此称为论题。

Church–Turing 论题讨论了图灵机能计算什么，接下来我们还关心完成计算需要多少时间。将计算效率也纳入考虑，就得到下面的推广形式。

论题 10.5 (推广的 Church–Turing 论题)**.** 对于现实可实现的确定性、非量子计算系统，若一次计算需要 T 步，则存在图灵机完成同样的计算，所需时间不超过 T 的某个固定多项式。

其中的多项式可以依赖于被模拟的系统，但不随输入改变。对允许随机选择的系统，通常相应地使用概率图灵机表述这一论题。这一主张也解释了多项式时间作为效率标准的意义。如果两个模型能够这样相互模拟，那么一个模型中的多项式时间算法，换到另一个模型中仍然只需多项式时间。

10.3.2 非确定性图灵机与 P、NP

给定输入后，确定性图灵机每走一步，都由当前状态和读到的符号唯一确定下一步的动作，因此整个计算过程只有一条路径。非确定性图灵机则允许同一个状态和符号对应多条转移规则。一次计算在每一步选取一条适用的规则继续执行，把所有可能选择都画出来，就得到图 10.2 这样的计算树。

树根表示图灵机刚读入这个输入时的初始构型。从根出发，每条适用的规则都引出一条边连接执行这条规则后的构型。从这些构型继续向下展开，每下降一层，图灵机就多走了一步。沿着图中的粗线往下看，图灵机经过三步到达接受状态。只要存在这样一条接受分支，图灵机就接受这个输入，无论其他分支是拒绝还是一直算下去。如果每条分支都停机并拒绝，图灵机就拒绝这个输入。

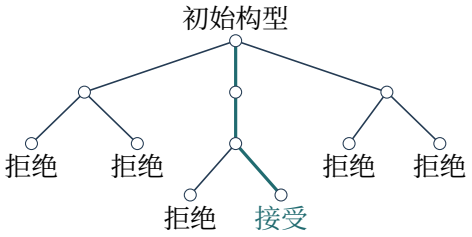


图 10.2 非确定性图灵机在一个固定输入上的计算树。粗线标出一条接受分支。

非确定性允许图灵机尝试不同的选择，是否因此就能解决确定性图灵机无法解决的问题？下面的定理说明，两者能够识别、判定的语言相同。

> **定理 10.6.** 每个非确定性图灵机识别的语言，都可以由一台确定性图灵机识别。若非确定性图灵机在每个输入上的所有分支都停机，则还存在确定性图灵机判定同一个语言。

证明. 非确定性图灵机每步只有有限种选择，全部计算过程形成一棵树。确定性图灵机可以按广度优先的顺序搜索这棵树，先检查执行一步后的所有情况，再检查两步后的情况，如此继续。只要发现一条接受分支，就接受输入。每层的结点数有限，因此任何有限长的接受分支最终都会被找到，得到的确定性图灵机便识别同一个语言。更进一步，若非确定性图灵机的所有分支都会停机，那么确定性图灵机也会在有限时间内找到所有分支，最终停机并给出正确答案。

具体模拟时，可以用第一条纸带保存输入，第二条纸带模拟当前分支的状态、磁头位置和纸带内容，第三条纸带记录广度优先搜索的进程，提示第二条纸带下一步应该模拟哪条路径。

多条纸带也能用一条纸带模拟，这一点比较显然，最简单的想法就是把纸带分成三个部分即可，具体实现超出这门课程范围，不再赘述。 □

还需要考虑上述模拟花费多少时间。若输入长度为 N，非确定性图灵机的每条分支都在 $T(N)$ 步内结束。当每步至多有 b 种选择时，计算树可能有 $1 + b + \cdots + b^{T(N)}$ 个结点，逐层模拟就可能需要指数时间。这样，非确定性图灵机只需执行多项式步的计算，按上述方法模拟时却可能耗费指数时间。因此，非确定性图灵机和确定性图灵机在可计算性上是等价的，但是在复杂度上可能大相径庭。这也引出了这一章最核心的定义。

> **定义 10.7.** P 是可被确定性图灵机在输入长度 N 的多项式时间内判定的语言类，NP 是可被非确定性图灵机在多项式时间内判定的语言类。

由于确定性图灵机每步只有一种选择，因此是非确定性图灵机的特殊情形，所以 $\mathsf{P} \subseteq \mathsf{NP}$。NP 中的 N 来自 non-deterministic，表示非确定性。相反，是否有 $\mathsf{NP} \subseteq \mathsf{P}$，也就是 $\mathsf{P} = \mathsf{NP}$，目前仍是计算复杂性理论中最著名的未解问题之一。

10.3.3　用证书描述接受计算

要在图中寻找 Hamilton 回路，我们可能需要尝试许多不同的顶点顺序。但如果有人已经给出一个顺序，只需检查每个顶点是否恰好出现一次，以及相邻顶点和首尾顶点是否有边，就能确认它是不是一条 Hamilton 回路。这个顺序就是一份**证书（certificate）**，检查它便能确认图中确实存在 Hamilton 回路。实际上，NP 中每个输入的肯定答案都可以用一份长度为多项式、能够快速验证的证书来说明。下面证明这一说法与非确定性图灵机的定义等价。

> **定理 10.8.** $L \in \mathsf{NP}$ 当且仅当存在多项式 p 和确定性多项式时间验证算法 V，使得
>
> $$x \in L \iff \exists y \in \{0,1\}^{\leqslant p(|x|)} : V(x,y) = 1.$$
>
> 这里的 y 称为证书。验证算法 V 对所有满足长度界的 (x,y) 都在多项式时间内停止。

证明. 先设 $L \in \mathsf{NP}$，取一台在多项式时间内判定 L 的非确定性图灵机 M。对于输入 x，只要记录每一步选择了哪条转移规则，就能重现一条计算分支，因此可以用这串规则编号作为证书 y。M 的规则总数固定，每个编号只需常数个比特，而每条分支的步数都有关于 $|x|$ 的多项式上界，所以证书长度也有多项式上界。验证算法从 M 在输入 x 上的初始构型出发，依次读取证书中的规则编号，检查规则是否适用于当前构型，并执行相应转移。只有所有转移都合法且最终到达接受状态，才通过验证。模拟的步数有多项式上界，因此验证也能在多项式时间内完成。当 $x \in L$ 时，至少有一条接受分支，它的规则序列就能通过验证。当 $x \notin L$ 时，没有接受分支，因此任何证书都无法通过验证。

反过来，假设存在这样的验证算法。非确定性图灵机先从 $0, 1, \ldots, p(|x|)$ 中选择证书长度 ℓ，再连续选择 ℓ 个比特，每次可选 0 或 1，组成字符串 y，最后运行 $V(x,y)$。每条分支所需时间都是多项式。如果存在合法证书，就有一条分支选中它并接受，如果不存在，所有分支都会被验证算法拒绝。这样就得到判定 L 的非确定性多项式时间算法。 □

回到前面的 Hamilton 回路。对于有 $n \geqslant 3$ 个顶点的简单无向图，先检查给出的列表是否恰好包含全部顶点各一次，再检查列表中相邻顶点以及首尾顶点之间是否都有边。检查顶点时，可以用一个布尔数组记录哪些顶点已经出现，遇到重复或不合法的编号就拒绝。顶点顺序只需 $O(n \log n)$ 位，邻接矩阵已经读入后，上述检查只需 $O(n)$ 次访问，连同读入图的时间仍是多项式。通过检查就说明这份证书给出了一条 Hamilton 回路。类似地，SAT 问题的证书是各变量的赋值，背包判定的证书是每件物品选或不选的 n 个比特，并且验证都只需要多项式时间。

10.3.4　co-NP

NP 要求对肯定答案有容易验证的证书。那么，否定答案是否也有这样的证书？为了讨论这类问题，我们对语言取补，并定义相应的复杂度类。对语言类 C，令 co-C $= \{L : \overline{L} \in \mathsf{C}\}$，也就是将 C 中每个语言的补集组成一个新的类。特别地，$L \in$ co-NP 表示 $\overline{L} \in \mathsf{NP}$，因此，当 $x \notin L$ 时，可以给出一份证书，证明 x 属于补语言。

例如，判断一个公式是否为永真式时，否定答案可以用一组使公式为假的赋值来证明。检查这组赋值只需计算公式的值，所以非永真式有多项式时间可验证的证书，永真式语言 TAUT 属于 co-NP。

用上一节的验证算法写出 $L \in$ co-NP 的条件，便是存在多项式 p 和确定性多项式时间算法 V，使

$$x \notin L \iff \exists y \in \{0,1\}^{\leqslant p(|x|)} : V(x,y) = 1.$$

这里 V 检查的是 x 不属于 L 的证书。将等价式两边同时取否定，得到

$$x \in L \quad \Longleftrightarrow \quad \forall y \in \{0,1\}^{\leqslant p(|x|)} : V(x,y) = 0.$$

也就是说，x 属于 L 时，所有试图证明它不属于 L 的证书都会被拒绝。

我们已经知道，确定性图灵机每个输入只有一条计算路径。若它总能停机，只需将接受和拒绝状态交换，就能判定补语言，所需时间也相同。因此 P 中语言的补语言仍属于 P，即 co-P = P。而非确定性图灵机有多条计算分支，交换接受和拒绝时就会遇到问题。例如，某个输入恰有两条分支，其中一条接受，一条拒绝，图灵机便接受这个输入。交换以后，两条分支仍然是一条接受、一条拒绝，整个输入依然被接受，因此得到的并不是判定原语言补集的图灵机。因此，确定性图灵机的做法无法直接用于非确定性图灵机。事实上，NP = co-NP 是否成立，目前仍是公开问题。

最后来看这些语言类之间的关系。若 $L \in$ P，则 L 和 $\overline{L}$ 都属于 P，因而也都属于 NP。前者说明 $L \in$ NP，后者说明 $L \in$ co-NP，所以

$$\mathsf{P} \subseteq \mathsf{NP} \cap \text{co-NP}.$$

若进一步假设 P = NP，那么对任意 $L \in$ co-NP，它的补语言属于 NP = P，再取一次补就知道 $L \in$ P。结合上面的包含关系，便得到 P = NP = co-NP。至于仅由 NP = co-NP 能否推出 P = NP，目前还没有答案。

10.4 归约与 NP 完全性

10.4.1 归约方向

有时一个新问题看起来很难，却能通过转换，交给已有算法求解。例如，把问题 A 的输入转成问题 B 的输入，然后运行求解 B 的算法。如果转换后的答案能够回答原问题，那么求解 B 的算法也就帮助我们解决了 A。复杂度理论用**归约（reduction）**描述这种关系。

下面先讨论判定问题，并要求转换前后的答案相同。注意，为了比较问题的多项式时间可解性，转换本身也应当在多项式时间内完成。

定义 10.9. 若存在确定性多项式时间可计算的函数 f，对每个字符串 x 都满足

$$x \in A \iff f(x) \in B,$$

则记作 $A \leqslant_P B$，称 A 可多项式归约到 B，也称 Karp 归约。



图 10.3 $A \leqslant_P B$：用 B 的判定方法解决 A。箭头表示算法执行顺序。

这个定义同时要求两个方向成立。原来答案为“是”的输入，转换后也必须为“是”；原来答案为“否”的输入，转换后也必须为“否”。这样，才能把 B 的答案直接用于 A。根据上述定义，不难理解 $A \leqslant_P B,\ B \in \mathsf{P} \implies A \in \mathsf{P}$。如果把输入原样输出，就得到 $A \leqslant_P A$。如果 $A \leqslant_P B$ 且 $B \leqslant_P C$，依次执行两个转换便得到 $A \leqslant_P C$，总时间仍是多项式。因此归约关系具有自反性和传递性。

定义 10.10. 如果每个 $L \in \mathsf{NP}$ 都满足 $L \leqslant_P B$，称 B 是 **NP-困难**（NP-hard）的。若同时 $B \in \mathsf{NP}$，称 B 是 **NP-完全**（NP-complete）的。其他语言类的难与完全可同样定义。

如果某个 NP 完全问题有多项式时间算法，那么每个 NP 问题都可以先归约到它，再调用这个算法求解，于是 $\mathsf{P} = \mathsf{NP}$。反过来，若 $\mathsf{P} = \mathsf{NP}$，NP 完全问题自然也有多项式时间算法。因此，只要解决一个 NP 完全问题的多项式时间可解性，就回答了 P 是否等于 NP。这也是为什么我们如此关心完全性。

有了一个已知的 NP 完全问题以后，证明新问题 B 的完全性就方便多了。先给出证书和验证算法，证明 $B \in \mathsf{NP}$，再选一个已知的 NP 完全问题 A，构造 $A \leqslant_P B$。对任意 $L \in \mathsf{NP}$，已有 $L \leqslant_P A$，接上新归约就得到 $L \leqslant_P B$。这也说明了归约方向为什么必须从已知问题指向待证明的问题：我们需要利用 B 的算法求解 A，才能把 A 的困难性传递给 B。

Richard M. Karp 在 1972 年的文章 *Reducibility among Combinatorial Problems* 中系统展示了许多组合问题之间的这种归约，Karp 归约也由此得名。沿着归约，我们可以从一个已知的完全问题出发，证明许多其他问题完全。不过，这还留下了一个必须先回答的问题：最初没有已知完全问题可用时，第一个 NP 完全问题是怎样证明的?

10.4.2　Cook–Levin 定理

我们选择公式可满足性作为归约链的起点。本节用 SAT 表示 CNF 公式可满足性问题，每个子句的长度不受限制。[1]给定变量赋值，逐个检查子句就能验证公式是否为真，因此 $\mathrm{SAT} \in \mathsf{NP}$。下面要证明的是，无论选择 NP 中的哪一个语言，都能将它归约到 SAT。这就是 Cook–Levin 定理。

定理 10.11. CNF 公式可满足性问题 SAT 是 NP 完全的。具体地，对每个语言 $L \in \mathsf{NP}$，都存在确定性多项式时间可计算的映射 $x \mapsto F_x$，使

$$x \in L \quad \Longleftrightarrow \quad F_x \in \mathrm{SAT}.$$

为了证明这一结论，我们从 NP 的定义出发。每个 $L \in \mathsf{NP}$ 都有一台在多项式步数内判定它的非确定性图灵机，所以判断 $x \in L$，就是判断这台图灵机是否存在一条接受 x 的计算分支。我们的想法是把这段计算写成表格，再用布尔公式约束表格中的内容，要求它从输入 x 开始，每一步都遵守图灵机规则，最后到达接受状态。如果能做到这一点，寻找接受分支就变成了寻找公式的满足赋值。

用表格记录计算

考虑一个很简单的语言：二进制串中至少出现一个 1。图灵机处于 q_0 时，读到 0 就向右移一格，读到 1 就接受，读到空白就拒绝。三个规则为

$$r_0 : (q_0, 0) \mapsto (q_0, 0, 1), \quad r_1 : (q_0, 1) \mapsto (q_{\mathrm{acc}}, 1, 0), \quad r_{\square} : (q_0, \square) \mapsto (q_{\mathrm{rej}}, \square, 0).$$

三元组最后的 $1, 0$ 分别表示向右移动和不移动。这台图灵机在长度 N 的输入上至多运行 $N + 1$ 步。后面要用布尔变量表示表格中的每个位置，因此需要先确定表格有多少行。我们按运行时间的上界

[1]CNF 公式要求每个子句都成立。如果一个子句也没有，就没有需要满足的条件，因此约定公式为真。如果某个子句中一个文字也没有，就无法让其中任何文字为真，因此约定这个子句为假，整个公式也为假。

确定行数，如果图灵机提前停机，就在剩余各行重复填写停机时的构型。这样，无论何时停机，都可以通过最后一行的状态判断是否接受。

输入 $x = 01$ 时，取运行步数的上界为 $T = 3$，计算如下，其中粗体表示磁头所在格。

时刻	状态	磁头位置	第 0 格	第 1 格	本步规则
0	q_0	0	**0**	1	r_0
1	q_0	1	0	**1**	r_1
2	q_{acc}	1	0	**1**	接受后的原地规则
3	q_{acc}	1	0	**1**	无下一步

从这张表可以看出，相邻两行之间只会改变图灵机状态、磁头位置和磁头正在访问的格子，其他格子的内容都保持不变。因此，检查一张表是否记录了合法的接受计算，可以分成三件事，分别检查起始行是否正确，相邻行是否符合转移规则，以及最后一行是否接受。对于非确定性图灵机，检查相邻两行时，只要有一条允许的转移规则能够把上一行变成下一行，就认为这一步合法。因此，同样可以逐行检查一张表是否记录了一条接受分支。

把计算表写成 CNF 公式

有了计算表，接下来就可以构造公式，完成 Cook–Levin 定理的证明。

证明. 固定 $L \in \mathsf{NP}$ 及判定它的非确定性图灵机 M，设输入长度为 N 时每条分支至多运行 $T = p(N) \geqslant \max\{N, 1\}$ 步。归约算法事先固定了 M，只有输入 x 会改变，因此图灵机的状态数、字母表大小及转移规则数都是常数。

为了让所有计算表都有 $T+1$ 行，我们把停机后的记录补齐到第 T 步。图灵机进入接受或拒绝状态后，后续各行都复制停机时的状态、磁头位置和纸带内容。磁头从第 0 格出发，每步最多移动一格，因此只需记录第 $-T$ 到第 T 格。

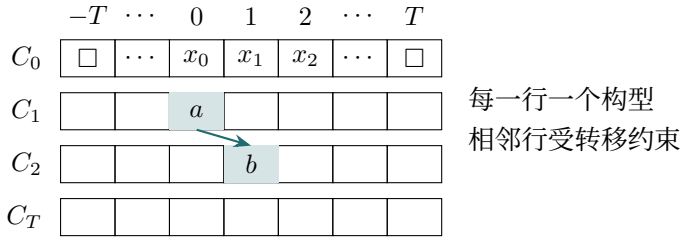


图 10.4 计算历史表的示意，其中阴影标出相邻时刻的磁头位置。

接下来为表格中的信息设置布尔变量，使一份赋值能够指定每个时刻的状态、磁头位置、纸带内容和采用的规则。各类变量的含义如下。状态、磁头位置和纸带内容都记录到时刻 T，因此前三类变量的时间下标为 $0 \leqslant t \leqslant T$。规则变量 $I_{t,r}$ 表示从时刻 t 到 $t+1$ 所用的规则，因此只需为 $t = 0, \ldots, T-1$ 设置。位置下标满足 $-T \leqslant h \leqslant T$，$q, a, r$ 分别遍历固定的状态、符号、规则集合。

变量	取真时的含义	变量	取真时的含义
$S_{t,q}$	时刻 t 的状态为 q	$B_{t,h,a}$	时刻 t 的第 h 格存放 a
$H_{t,h}$	时刻 t 的磁头在第 h 格	$I_{t,r}$	第 t 步选择规则 r

对一组变量 $z_1,\dots,z_k$，“恰有一个为真”可以直接写为

$$\mathrm{One}(z_1,\dots,z_k)=(z_1\vee\cdots\vee z_k)\ \wedge\bigwedge_{1\leqslant i<j\leqslant k}(\neg z_i\vee\neg z_j).$$

第一个子句要求至少一个变量为真，后面的子句则要求任意两个变量不能同时为真，合起来正好表示恰有一个为真。用这一公式，就可以逐项写出计算表需要满足的条件。

唯一性。对每个 t，状态变量和磁头变量恰有一个为真。对每个 (t,h)，符号变量恰有一个为真。对每个 $t<T$，规则变量恰有一个为真。于是每一行确实描述一个构型，每一步确实选择一条规则。

起点与终点。首先加入单文字子句 S_{0,q_0} 和 $H_{0,0}$，然后对于 $0\leqslant h<N$ 加入 B_{0,h,x_h}，其他位置加入 $B_{0,h,\square}$，最后加入 $S_{T,q_{\mathrm{acc}}}$。

被选择的规则。设规则 r 为 $(q,a)\mapsto(q',a',d)$，其中 $d\in\{-1,0,1\}$ 表示磁头移动方向。对每个 $t<T$，加入 $\neg I_{t,r}\vee S_{t,q}$ 和 $\neg I_{t,r}\vee S_{t+1,q'}$。再对每个 h 加入

$$\neg I_{t,r}\vee\neg H_{t,h}\vee B_{t,h,a},\quad \neg I_{t,r}\vee\neg H_{t,h}\vee B_{t+1,h,a'},\quad \neg I_{t,r}\vee\neg H_{t,h}\vee H_{t+1,h+d}.$$

这些子句使用了 $x\Rightarrow y$ 与 $\neg x\vee y$ 以及 $(x\wedge y)\Rightarrow z$ 与 $\neg x\vee\neg y\vee z$ 的等价关系，据此不难理解其含义。例如最后一条要求在选择规则 r 且磁头位于 h 时，下一行的磁头移到 $h+d$。如果没有选择 r，则 $\neg I_{t,r}$ 为真，这组子句自然满足。

没有访问的位置。对每个 $t<T,h,a$ 加入 $H_{t,h}\vee\neg B_{t,h,a}\vee B_{t+1,h,a}$。当磁头不在 h 时，它要求旧符号继续保留。

把上述子句全部用合取连接，得到公式 F_x。接下来要证明，F_x 可满足当且仅当图灵机接受 x。如果图灵机接受 x，我们沿一条接受分支记录实际状态、磁头、纸带和规则，把表示这些实际值的变量设为真，显然全部子句同时为真，故 F_x 可满足。反之，若 F_x 可满足，取一组满足赋值。由唯一性约束，每一行都确定了一个状态、一个磁头位置和各格的符号，因而可以从赋值中读出一列构型 $C_0,\dots,C_T$。起始条件保证 C_0 正是输入 x 对应的起始构型。被选择的规则约束和没有访问的位置约束保证 C_{t+1} 恰好是从 C_t 执行规则 r 得到的构型。这样逐行检查，就得到一条从输入 x 开始的合法计算分支。最后一行的状态由子句 $S_{T,q_{\mathrm{acc}}}$ 保证为接受状态，所以图灵机接受 x。两个方向合起来便得到 $x\in L\iff F_x$ 可满足。

最后检查构造出的公式有多长。构型变量有 $O(T^2)$ 个，规则变量有 $O(T)$ 个，故唯一性约束共 $O(T^3)$ 个子句，其他约束合计 $O(T^2)$ 个子句。变量编号占 $O(\log T)$ 位，故完整编码长 $O(T^3\log T)$，按前面列出的约束逐项生成子句，就能在多项式时间内构造这个公式。$T=p(N)$ 是多项式，这正是所需的归约。 □

10.4.3 从 SAT 到 3-SAT

Cook–Levin 定理已经证明 SAT 是 NP 完全问题。接下来我们要证明，即使限制每个子句至多有三个文字，得到的 3-SAT 仍然是 NP 完全问题。给定赋值后，3-SAT 同样可以在多项式时间内验证，因此只需证明 SAT $\leqslant_P$ 3-SAT。

转换时，至多含三个文字的子句已经符合要求，可以直接保留。我们需要处理的是更长的子句，为此可以引入辅助变量，将它拆成若干个至多含三个文字的子句。具体地，对于 $C=u_1\vee\cdots\vee u_m$，其中 $m>3$，引入仅供这个子句使用的新变量 $z_1,\dots,z_{m-3}$，将 C 替换为以下子句的合取：

$$u_1\vee u_2\vee z_1,\quad \neg z_j\vee u_{j+2}\vee z_{j+1}(1\leqslant j\leqslant m-4),\quad \neg z_{m-3}\vee u_{m-1}\vee u_m.$$

先看原子句不成立的情况。若所有 u_i 都为假，第一条子句要求 $z_1 = 1$，随后各条子句依次要求 $z_2, z_3, \ldots$ 都为 1。于是最后一条的三个文字全部为假，新公式也无法满足。再看原子句成立的情况。我们让 z_j 记录前面是否还没有遇到真文字，具体取 $z_j = 1$ 当且仅当 $u_1, \ldots, u_{j+1}$ 全为假。第一条子句因此成立。对中间的子句，如果 $z_j = 0$，它的第一个文字已经为真，如果 $z_j = 1$，那么遇到真文字 u_{j+2} 时该子句成立，没有遇到时则由 $z_{j+1} = 1$ 保证成立。最后一条也成立，因为前面已经出现真文字时有 $z_{m-3} = 0$，否则原子句中的真文字必在最后两个位置。因此，原子句成立恰好等价于辅助变量存在一种取值，使所有新子句都成立。

对每个长子句分别进行上述拆分，不同子句使用不同的辅助变量，再将得到的子句合取，就得到所需的 3-CNF 公式。记原公式为 F，转换后的公式为 F'。若 F 可满足，就保留原变量的满足赋值，并分别为每个长子句新增的辅助变量选择合适的取值，使拆分后的子句全部成立。由于不同子句使用不同的辅助变量，这些选择互不冲突，故得到 F' 的满足赋值。反之，若 F' 有满足赋值，去掉其中辅助变量的取值，剩下的原变量取值就能使 F 的每个子句成立。因此，F 可满足当且仅当 F' 可满足。

于是，求解 SAT 可以分成两步：先将输入公式 F 转换为 F'，再判断这个 3-SAT 实例是否可满足，得到的答案也就是原 SAT 实例的答案。由于新公式的文字总数与原公式的文字总数成线性关系，归约可以在多项式时间内完成。因此 SAT $\leqslant_P$ 3-SAT，根据归约的传递性可知 3-SAT 是 NP 困难的，再结合它属于 NP，便得到 3-SAT 是 NP 完全的。[2]

10.4.4 从判定恢复证书

前面讨论的是如何判断答案是否存在。如果答案存在，我们能不能进一步把它找出来？以 SAT 为例，先问公式是否可满足，再尝试把第一个变量固定为 0，询问剩下的公式是否可满足。如果可以，就保留这个取值，否则将它固定为 1。逐个确定变量，就能得到一份满足赋值。因为固定变量以后仍是 SAT 问题，这种方法称为 SAT 的自归约。

对于一般的 NP 问题，也可以逐位寻找证书。下面证明，若 P = NP，整个过程可以在多项式时间内完成。设证书是长度不超过 $p(|x|)$ 的二进制串，其中 p 是多项式。给定已经确定的开头 u，我们要判断是否存在以 u 开头的合法证书。这个判定问题属于 NP，因为给出完整证书后，只需检查它的长度、开头和合法性。因此，在 P = NP 的假设下，这个问题可以在多项式时间内解决。因此我们可以从空串开始，先判断是否存在证书，不存在就报告无解。若存在，就逐位确定证书，每次尝试在末尾添上 0，判断能否补成合法证书。可以就保留 0，否则选 1，因为当前开头能够补全，下一位必有一种可行的选择。每次添完一位都检查当前字符串是否已是合法证书，是就返回，空串本身合法时也直接返回。这样至多确定 $p(|x|)$ 位，每一步的判断和验证都只需多项式时间，因此总时间也是多项式。

10.5 几个具体的归约

10.5.1 3-SAT、团与顶点覆盖

有了 SAT 和 3-SAT 的完全性，接下来就可以证明其他问题也同样困难。关键是把满足赋值转成目标问题中的一个解，并保证目标问题的解也能反过来给出满足赋值。下面先看怎样用图来表达子句之间的约束。

[2]如果进一步要求归约前后的解的数量相同，就称为**约俭归约（parsimonious reduction）**。上面的拆分可能增加满足赋值的数量。例如，$a \vee b \vee c \vee d$ 被拆成 $(a \vee b \vee z) \wedge (\neg z \vee c \vee d)$。原来的四个变量全取 1 时是一份满足赋值，拆分后 z 取 0 或 1 都能使新公式成立，就对应两份满足赋值。因此，这个构造可以用于判断是否有解，却没有保留解的数量。

在简单无向图中，如果一组顶点中任意两个不同顶点之间都有边，就称这组顶点构成一个**团（clique）**。团问题（CLIQUE）给定一个图和整数 k，要求判断图中是否存在至少包含 k 个顶点的团。给出一组顶点后，我们只需检查顶点数是否至少为 k，以及任意两个不同顶点之间是否都有边，就能判断它是否符合要求。这些检查都可以在多项式时间内完成，因此 CLIQUE $\in$ NP。

接下来要证明团问题是 NP 完全的。为此，我们将 3-SAT 归约到团问题。给定含 m 个子句的公式 F，要构造一个图，使 F 可满足当且仅当图中存在至少包含 m 个顶点的团。公式可满足，就意味着可以从每个子句中各选一个文字，让这些文字同时为真。选择时唯一可能的冲突是同时选中了某个变量及其否定。我们可以用顶点表示这些选择，用边表示两个选择可以同时成立，这样一组两两相连的顶点就对应一组互不冲突的选择。具体地，对 $F = C_1 \wedge \cdots \wedge C_m$，每个子句中的每次文字出现都建立一个顶点。同一文字出现在不同子句中时，分别建立顶点，以便记录它来自哪个子句。两个顶点之间连边，当且仅当它们来自不同子句，且对应的文字不互为否定。最后令团问题的参数 $k = m$。

若 F 有满足赋值，就从每个子句中各选一个为真的文字。选出的文字不会互相否定，所对应的顶点又来自不同子句，所以这些顶点两两相邻，组成大小为 m 的团。反过来，假设图中有一个至少包含 m 个顶点的团，取其中任意 m 个顶点。同一子句的顶点之间没有边，因此团中至多有一个顶点来自同一子句，一共只有 m 个子句，所以每个子句恰好有一个顶点入选。将这些顶点对应的文字都设为真。由于团中不会同时出现互相否定的文字，这些取值不会冲突。其余变量任意取值，每个子句都有一个真文字，于是 F 可满足。

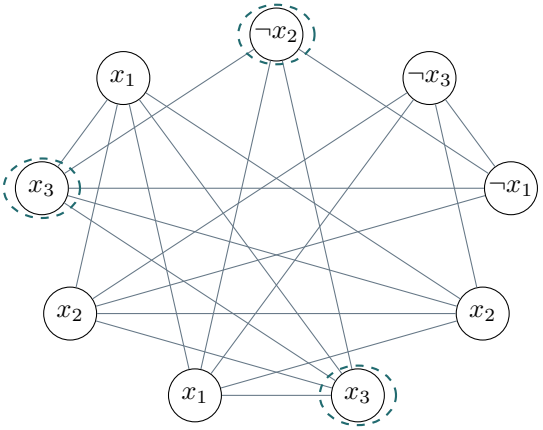


图 10.5 公式 $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$ 对应的图。虚线圈出的三个顶点分别对应 $\neg x_2, x_3, x_3$，它们构成一个团。取 $x_2 = 0, x_3 = 1$，这三个文字便同时为真，x_1 可任取。

构造出的图至多有 $3m$ 个顶点，所以归约可以在多项式时间内完成。公式有空子句时，该子句没有对应顶点，图中便不可能选出所需的团；公式没有子句时，得到空图和 $k = 0$，空集满足要求。结合前面的双向证明与证书验证，得到 CLIQUE 是 NP 完全问题。

接下来利用团问题的完全性，证明顶点覆盖问题也是 NP 完全的。在图 $G = (V, E)$ 中，如果每条边都至少有一个端点属于 $S \subseteq V$，就称 S 为顶点覆盖。判定问题要求判断是否能用至多 k 个顶点覆盖所有边。给出 S 后，检查其大小并逐条检查边，就能在多项式时间内验证，因此顶点覆盖问题属于 NP。还需构造团到顶点覆盖的归约。一个集合是顶点覆盖，恰好要求它以外的顶点之间没有边，否则那条边就无法被覆盖。而团要求选中的顶点之间都有边，因此可以先把有边和无边的关系交换，再取顶点集合的补集。

为此，构造 G 的**补图（complement graph）** $\overline{G}$，保留所有顶点，将原来不相邻的两个不同顶点连边，并去掉原有的边。则对任意 $S \subseteq V$，S 是 G 的团 $\iff$ S 在 $\overline{G}$ 中没有内部边 $\iff$ $V \setminus$

S 覆盖 $\overline{G}$ 的所有边。因此，将原图 G 换成补图 $\overline{G}$，并把要求的团大小 k 换成覆盖大小的上限 $|V|-k$，就得到了顶点覆盖问题的实例。原图有至少含 k 个顶点的团，当且仅当补图有至多含 $|V|-k$ 个顶点的覆盖。显然归约过程只需要多项式时间，因此顶点覆盖问题是 NP 完全的。

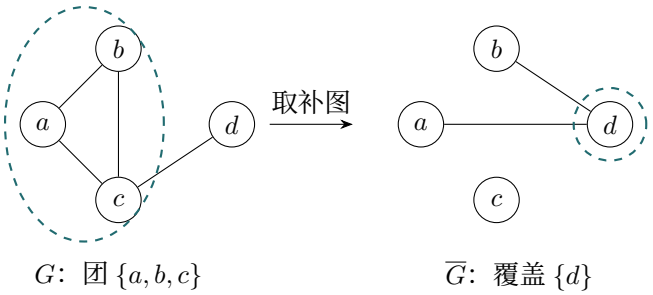


图 10.6 团归约到顶点覆盖的例子。

10.5.2 2-SAT 的多项式时间算法

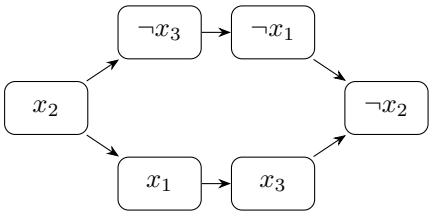
前面从 3-SAT 出发，证明了几个问题的 NP 完全性。如果把公式中每个子句的文字数进一步限制为至多两个，就得到 2-SAT。这个限制使我们能够用图来求解，并得到一个多项式时间算法。下面从子句 $a \vee b$ 入手：为了让它成立，a 为假时就必须让 b 为真，b 为假时也必须让 a 为真。我们可以把这两条蕴含关系表示为有向边。

对每个变量 x_i，建立两个顶点 $x_i, \neg x_i$。一个子句 $a \vee b$ 等价于 $(\neg a \Rightarrow b) \wedge (\neg b \Rightarrow a)$，因此加入有向边 $\neg a \to b$ 与 $\neg b \to a$。单文字子句 a 可以写成 $a \vee a$，对应边 $\neg a \to a$。如果公式含有空子句，就直接判定为不可满足。按这一规则建图后，图中边 $u \to v$ 就表示如果 u 为真，v 也必须为真，路径表示连续的蕴含。如果 x 与 $\neg x$ 能相互到达，令任何一个为真都会迫使另一个为真，因而不可能有满足赋值。这给出了必要条件：它们不能在同一个强连通分量中。如果将每个强连通分量缩成一点，得到的图称为缩点图，它没有有向环，否则环上的分量本应合并。

接下来证明，只要每个变量与它的否定都不在同一个强连通分量中，就一定能找到满足赋值。先对缩点图进行拓扑排序，按排序结果依次给各分量编号，使每条跨分量的边都从编号较小的分量指向编号较大的分量。记 $c(a)$ 为文字 a 所在分量的编号。同一分量内的顶点编号相同，因此对于图中任意一条边 $a \to b$，都有 $c(a) \leqslant c(b)$。赋值时，在 x_i 和 $\neg x_i$ 中，将分量编号较大的那个文字设为真，另一个设为假。由于它们属于不同分量，编号一定不同，所以每个变量的取值都能确定，即 $x_i = 1 \iff c(x_i) > c(\neg x_i)$。下面验证，这样赋值能够满足图中的所有蕴含关系。

假设某条边 $a \to b$ 对应的蕴含不成立，也就是 a 为真而 b 为假。按照赋值规则，a 为真意味着 $c(\neg a) < c(a)$，b 为假意味着 $c(b) < c(\neg b)$。另一方面，建图时每条边 $a \to b$ 都与 $\neg b \to \neg a$ 成对出现。由拓扑编号的性质，这两条边分别给出 $c(a) \leqslant c(b)$ 和 $c(\neg b) \leqslant c(\neg a)$。把这四个不等式连起来，就得到 $c(\neg a) < c(a) \leqslant c(b) < c(\neg b) \leqslant c(\neg a)$，产生矛盾。因此，每条边对应的蕴含都成立。结合前面的必要性可知，2-SAT 可满足当且仅当每个变量与它的否定都属于不同的强连通分量。

强连通分量可以用 Kosaraju 算法在 $O(|V|+|E|)$ 时间内求出，具体可参考《算法导论》第三版第 22.5 节。求得各分量后，对缩点图进行拓扑排序也只需线性时间。上述归约显然也可以在多项式时间内完成，因此 2-SAT 可以在多项式时间内求解。



拓扑编号可取 $x_2, \neg x_3, \neg x_1, x_1, x_3, \neg x_2$

图 10.7　$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ 的蕴含图。每点自成分量，按次序赋值可得 $x_1 = 1, x_2 = 0, x_3 = 1$。

10.5.3　Hamilton 回路与旅行商

假如一位旅行商要到若干座城市各访问一次，最后回到出发地，怎样安排顺序才能使总路程最短？这就是**旅行商问题（Traveling Salesman Problem，TSP）**。将城市看作顶点，城市之间的路程看作边权，就得到一个带非负整数边权的完全无向图。它的判定版本再给出一个整数 K，要求判断是否存在经过每个顶点恰好一次、最后回到起点且总权重不超过 K 的回路。

证明 TSP 的困难性时，从 Hamilton 回路问题出发会更直接，因为两者都要求经过每个顶点恰好一次并回到起点。Hamilton 回路问题已经被证明是 NP 完全的，下面以这一结论为基础，证明 TSP 的判定版本也是 NP 完全的。首先，给出一条路线后，检查它是否恰好访问每个顶点一次并返回起点，再计算总权重是否超过 K，就能在多项式时间内验证，因此 TSP 属于 NP。

接下来将 Hamilton 回路问题归约到 TSP。给定原图 $G = (V, E)$，我们把缺少的边补齐，得到完全图 G'。为了让符合要求的路线只使用原图中的边，将原有边的权重设为 1，补入边的权重设为 2，并把总权重上限设为 $|V|$。这样，一条回路只要用了补入的边，总权重就会超过上限。具体地，令

$$w(u,v) = \begin{cases} 1, & \{u,v\} \in E, \\ 2, & \{u,v\} \notin E, \end{cases} \qquad K = |V|.$$

G 的 Hamilton 回路在 G' 中全部使用权为 1 的边，总权恰为 $|V|$。反过来，G' 的回路有 $|V|$ 条边，每条至少为 1，若总权至多为 $|V|$，每条就必须都是 1，故全部是 G 的边。因此，原图 G 有 Hamilton 回路，当且仅当构造出的完全图 G' 中存在一条经过每个顶点恰好一次并回到起点、总权重不超过 $|V|$ 的回路。归约枚举点对输出权重即可，构造时间 $O(|V|^2)$。

这个构造还满足距离的三角不等式：任一边至多为 2，两条边之和至少为 2。下一讲会利用三角不等式为 TSP 设计近似算法，在多项式时间内求出与最优值相差有限倍的解。

10.5.4　从 3-SAT 到子集和

前面通过归约证明了几个图问题的困难性。下面考察一个与整数有关的子集和（SUBSET-SUM）问题：给定非负整数 $a_1, \ldots, a_r$ 和目标 B，判断能否从中选出若干项，使它们的和恰好为 B。每项最多选一次，数值相同但下标不同的两项可以分别选择。列出选中项的下标并求和，就能在多项式时间内验证答案，所以子集和属于 NP。

下面证明子集和也是 NP 完全问题。我们将 3-SAT 公式转成一组整数和目标数，使公式有满足赋值当且仅当这些整数中有若干项的和等于目标数。设原公式有 n 个变量、m 个子句。我们先为每个变量 x_i 准备两个整数 t_i 和 f_i，选 t_i 表示让 x_i 为真，选 f_i 表示让它为假。接下来要设计这些整数和目标数，使凑出目标数恰好要求每个变量选定一个值，而且这个赋值满足全部子句。

每个变量都必须取一个确定的真值，所以选出的整数中，t_i 和 f_i 必须有且只有一个。我们让求和等于目标数这一要求自动保证这一点。可以像列竖式做加法那样，把所有整数的十进制数位对齐，并为每个变量指定一个数位。在 x_i 对应的数位上，t_i 和 f_i 都写 1，其余整数都写 0，目标数也写 1。暂且假设求和时没有进位，后面构造完成后，我们再验证加法确实不会产生进位。如果两者都不选，这一位的和就是 0，两者都选则是 2，只有恰选一个才能得到目标数要求的 1。

例如，考虑公式 $F = C_1 \wedge C_2$，其中 $C_1 = x_1 \vee \neg x_2$，$C_2 = \neg x_1 \vee x_2 \vee x_2$。根据上面的讨论，我们可以列出下表。要使第一列的和为 1，就必须从 t_1, f_1 中选一个，第二列同理。

	x_1	x_2
t_1	1	0
f_1	1	0
t_2	0	1
f_2	0	1
目标	1	1

不过，到这里我们只保证了每个变量都有取值，还需要让这个取值满足两个子句。为此，在右边增加两列，分别对应 C_1 和 C_2。$C_1 = x_1 \vee \neg x_2$ 中，取 $x_1 = 1$ 会使文字 x_1 为真，取 $x_2 = 0$ 会使文字 $\neg x_2$ 为真，所以在 t_1, f_2 的 C_1 列各填 1，其余填 0。$C_2 = \neg x_1 \vee x_2 \vee x_2$ 中，取 $x_1 = 0$ 会使一个文字为真，取 $x_2 = 1$ 则会使两个文字为真，所以在 f_1 的 C_2 列填 1，在 t_2 的这一列填 2，其余填 0。这样补完后，四个数依次为 $1010, 1001, 0102, 0110$。选择 t_1, t_2 时，和为 1112，后两位的 1 和 2 就分别表示两个子句中真文字的数量。

子句只要有一个文字为真就成立，因此对应列的和为 $1, 2, 3$ 都应当允许，为 0 则应当排除。但子集和要求凑出一个固定的目标数，怎样才能让这三种情况都达到同一个目标呢？我们把每个子句列的目标设为 4，再提供两个整数，分别用来给这一列加 1 或加 2。例如，给 C_1 提供 0010 和 0020，给 C_2 提供 0001 和 0002。原来是 1，就把两个数都选上，原来是 2 或 3，就分别只选加 2 或加 1 的数。原来是 0 时，即使两个数都选，也只能得到 3，无法达到目标 4。如下表所示，将给 C_j 补数的两个整数记为 s_j, s'_j，表中前四列展示每个整数的数位，最后一列写出这个整数，目标数为 1144。

	x_1	x_2	C_1	C_2	整数
t_1	1	0	1	0	1010
f_1	1	0	0	1	1001
t_2	0	1	0	2	102
f_2	0	1	1	0	110
s_1	0	0	1	0	10
s'_1	0	0	2	0	20
s_2	0	0	0	1	1
s'_2	0	0	0	2	2
目标	1	1	4	4	1144

取满足赋值 $x_1 = x_2 = 1$，就应选 $t_1 = 1010$ 和 $t_2 = 102$，相加得到 1112。前两位已经达到目标，后两位分别是 1 和 2，恰好表示 C_1 中有一个真文字，C_2 中有两个真文字。为了把后两位都补到 4，再选 $s_1 = 10$、$s'_1 = 20$ 和 $s'_2 = 2$，便有 $1010 + 102 + 10 + 20 + 2 = 1144$。

对于一般的公式，也按同样的方法构造。前 n 列对应变量，t_i, f_i 在变量 x_i 的那一列都填 1，其余变量列填 0。后 m 列对应子句，在 C_j 对应的列中，t_i 填 x_i 的出现次数，f_i 填 $\neg x_i$ 的出现次数。每个子句再配两个整数 s_j, s'_j，只在这一列分别填 1、2，其余列填 0。目标数的前 n 位全为 1，后 m 位全为 4。

上面的逐列分析依赖于加法不发生进位，现在验证这一点。即使把全部整数都选上，每个变量列的和也只有 2。每个子句至多有三个文字，因此该子句列中全部 t_i, f_i 的数位之和至多为 3，再加上 s_j, s'_j 提供的 $1 + 2$，也至多为 6。各列都小于 10，所以任意选数方案都不会产生进位。

若原公式有满足赋值，就按赋值为每个变量选 t_i 或 f_i。变量列的和都是 1，子句列的和都是 $1, 2, 3$ 中的一个，再按前面的方法补到 4，就得到和等于目标数的子集。反过来，若选出的数恰好凑出目标，每个变量列都要求 t_i, f_i 恰选一个，由此确定一个赋值。每个子句列单靠两个补充整数最多只能得到 3，因此要达到目标 4，所选的 t_i, f_i 必须使这一列的和至少为 1，即该子句至少有一个文字为真。所以这个赋值满足原公式。空子句中没有文字，其对应列无法凑到 4，也符合这一结论。

整个构造生成 $2n+2m$ 个整数，每个整数至多有 $n+m$ 个十进制位，转为二进制后也只需 $O(n+m)$ 位。填写这些数位并转为二进制都能在多项式时间内完成，因而得到了 3-SAT $\leqslant_P$ SUBSET-SUM。结合子集和属于 NP，可知它是 NP 完全问题。

上面的归约证明了子集和是 NP 完全问题。另一方面，第八讲的动态规划思路也能用于求解子集和：依次考虑每个数，记录 $0, \ldots, B$ 中哪些和能够由已经考虑的数凑出，至多需要 $O(nB)$ 次表项更新，其中 n 为整数个数。因此这个算法属于伪多项式算法。如果 B 不超过 n 的某个固定次幂，上述动态规划就能在输入长度的多项式时间内完成。对于其他含整数参数的问题，也可以问类似的问题：限制输入数值以后，原来的困难性是否仍然存在？若将所有数值限制在物品数、顶点数等对象数量的某个多项式范围内，问题仍然 NP 困难，就称它为**强 NP 困难（strongly NP-hard）**问题。在 P $\neq$ NP 的假设下，子集和不具有这种强困难性，因为数值受上述限制时，它已有多项式时间算法。子集和通常被称为**弱 NP 完全（weakly NP-complete）**问题。0–1 背包的判定版本也是如此，习题 10.7 将讨论它的归约及数值受限时的算法。

10.6　补充材料

10.6.1　时间与空间复杂度类

到这里，我们主要讨论了哪些问题能在多项式时间内求解，以及如何用归约比较问题的困难程度。接下来进一步考察计算资源的影响。如果允许指数时间，能够解决哪些问题？如果只限制存储空间，又会得到怎样的结论？

指数时间复杂度类

前面定义的 P 要求问题能由确定性图灵机在多项式时间内判定，NP 则要求问题能由非确定性图灵机在多项式时间内判定，下面将多项式时间推广到一般的时间界。我们用 N 表示输入长度，用 $t(N)$ 表示给定的时间界。如果一个语言能由确定性图灵机在 $O(t(N))$ 时间内判定，就说它属于 DTIME$(t(N))$。对于非确定性图灵机，仍沿用前面的接受规则，只要有一条计算分支接受，就接受输

入。我们要求每条分支都在 $O(t(N))$ 步以内停机，能够这样判定的语言组成 $\mathsf{NTIME}(t(N))$。让 $t(N)$ 依次取 $N, N^2, N^3, \ldots$，将相应的语言类取并集，就得到 P 和 NP。如果将时间界改为 2^{N^k}，其中 k 可以是任意固定正整数，再将相应的类取并集，就得到指数时间类 EXP 和 NEXP。

$$\mathsf{P} = \bigcup_{k\geqslant 1} \mathsf{DTIME}(N^k), \qquad \mathsf{NP} = \bigcup_{k\geqslant 1} \mathsf{NTIME}(N^k),$$
$$\mathsf{EXP} = \bigcup_{k\geqslant 1} \mathsf{DTIME}(2^{N^k}), \qquad \mathsf{NEXP} = \bigcup_{k\geqslant 1} \mathsf{NTIME}(2^{N^k}).$$

对于一个 NP 问题，判定它的非确定性图灵机每步只有常数种选择，每条分支至多有多项式步。因此，全部分支的数量至多为 $2^{N^{O(1)}}$。确定性图灵机逐条模拟这些分支，只要发现一条接受分支就接受，全部失败后拒绝，总时间仍为 $2^{N^{O(1)}}$。这就证明了 $\mathsf{NP} \subseteq \mathsf{EXP}$。确定性图灵机也可以看成每步只有一种选择的非确定性图灵机，所以还有 $\mathsf{EXP} \subseteq \mathsf{NEXP}$。

定理 10.12. 如果 $\mathsf{EXP} \neq \mathsf{NEXP}$，则 $\mathsf{P} \neq \mathsf{NP}$。

证明. 我们证明逆否命题，即假设 $\mathsf{P} = \mathsf{NP}$，证明 $\mathsf{EXP} = \mathsf{NEXP}$。取任意 $L \in \mathsf{NEXP}$，设判定它的非确定性图灵机在长度为 N 的输入上至多运行 2^{N^k} 步。要使用 $\mathsf{P} = \mathsf{NP}$ 这一假设，需要先得到一个属于 NP 的语言。我们可以在原输入后面补上很长的一段字符，让原来的指数时间变成关于新输入长度的多项式时间。具体地，定义

$$L' = \{x\#1^{2^{|x|^k}} : x \in L\},$$

其中 # 用来分隔原输入和补上的 1。这种通过加长输入改变时间界的做法称为**填充（padding）**。

给定一个字符串，先检查它是否具有上述格式，包括末尾 1 的个数是否正确。格式正确后，在 x 上模拟原来的非确定性图灵机。新输入的长度至少为 $2^{|x|^k}$，所以格式检查和每条分支的模拟都能在新输入长度的多项式时间内完成。于是 $L' \in \mathsf{NP}$，由假设可知 $L' \in \mathsf{P}$。

设判定 L' 的确定性算法对长度为 M 的输入需要 $O(M^c)$ 时间。现在要判断 $x \in L$，可以先写出 $x\#1^{2^{|x|^k}}$，再调用这个算法。因为 $M = |x| + 1 + 2^{|x|^k}$，写出输入和运行算法合计需要 $2^{O(|x|^k)}$ 时间。因此 $L \in \mathsf{EXP}$，得到 $\mathsf{NEXP} \subseteq \mathsf{EXP}$。结合已有的 $\mathsf{EXP} \subseteq \mathsf{NEXP}$，命题得证。 □

空间复杂度与 Savitch 定理

前面一直用运行时间衡量算法的效率，接下来我们看看存储空间。计算空间复杂度时，关心的是算法运行过程中最多同时占用多少存储空间。例如，深度优先搜索虽然可能遍历很多条分支，却可以在回溯时释放已经不再需要的数据，将这部分空间留给后面的搜索。因此，一个算法即使运行很久，也可能只需要较少的存储空间。注意，这里我们计算的是输入之外额外需要的存储空间。在图灵机上，我们将输入保存在只读纸带上，另用工作带保存计算中的数据，只统计工作带使用的格子数。

能由确定性图灵机使用 $O(s(N))$ 工作空间判定的语言，组成 $\mathsf{DSPACE}(s(N))$。将图灵机换成非确定性的，并要求每条计算分支都只使用 $O(s(N))$ 工作空间，就得到 $\mathsf{NSPACE}(s(N))$。两者都要求图灵机最终停机。将允许的空间取为任意多项式，便得到

$$\mathsf{PSPACE} = \bigcup_{k\geqslant 1} \mathsf{DSPACE}(N^k), \qquad \mathsf{NPSPACE} = \bigcup_{k\geqslant 1} \mathsf{NSPACE}(N^k).$$

非确定性图灵机只要有一条计算分支接受，就接受输入。如果改用确定性图灵机，就需要自己寻找这样的分支。逐条尝试当然可以，但怎样搜索才能尽量少用存储空间？下面的 **Savitch 定理**说明，在下面的条件下，原来使用 $s(N)$ 空间的非确定性计算，可以用 $O(s(N)^2)$ 空间的确定性计算完成。

定理 10.13. 若 $s(N) \geqslant \log(N+2)$ 且空间可构造，则

$$\mathsf{NSPACE}(s(N)) \subseteq \mathsf{DSPACE}(s(N)^2).$$

定理中的**空间可构造（space-constructible）**要求，图灵机能够根据输入长度 N，在工作带上依次标记恰好 $s(N)$ 个格子，而且完成这件事所需的全部工作空间也是 $O(s(N))$。这些标记相当于划出可用的存储范围，计算标记数量、控制标记过程所用的空间也要计算。例如，若 $s(N) = N^2$，可以先计算 N^2，再用计数器控制，依次标记这么多个格子。计数器只需 $O(\log N)$ 位，加上标记占用的 N^2 个格子，总空间为 $O(N^2)$，所以 N^2 是空间可构造的。常用的正整数次幂空间界也满足这一条件。

证明. 固定原来的非确定性图灵机和输入，将每个合法构型看成一个顶点。如果从构型 u 执行一步可以得到构型 v，就连一条有向边 $u \to v$，这样得到的图称为**构型图（configuration graph）**。一条计算分支对应图中的一条路径，所以我们要判断的就是，从起始构型能否走到某个接受构型。

要搜索这张图，先确定路径最多需要检查到多长。原图灵机只使用 $O(s(N))$ 个工作格，纸带内容可以用 $O(s(N))$ 位表示。描述一个构型还要写明当前状态和磁头位置，状态只需 $O(1)$ 位，工作带磁头的位置需要 $O(\log s(N))$ 位，输入带磁头的位置需要 $O(\log N)$ 位，故这些信息合计仍为 $O(s(N))$ 位。将编码统一为 $b = O(s(N))$ 位，只有 2^b 种二进制串，因此合法构型也至多有 2^b 个。如果接受构型可达，就可以删去路径中重复经过同一顶点的部分，得到一条不重复顶点的路径。这样的路径长度小于 2^b，所以检查长度至多为 2^b 的路径已经足够。

直接保存这样一条路径可能占用太多空间。我们改用分治，只判断路径是否存在。递归的参数是两个构型 u, v 和整数 $k \geqslant 0$，要回答的问题是，从 u 到 v 是否存在长度至多为 2^k 的路径。递归开始时取 $k = b$，对应刚才得到的路径长度上限 2^b。当 $k = 0$ 时，长度上限为 $2^0 = 1$。如果 $u = v$，不经过任何边就已到达，回答存在。否则，检查是否有边 $u \to v$，也就是依次尝试在 u 中适用的转移规则，看执行一步后能否得到 v。有这样的规则就回答存在，没有则回答不存在。当 $k > 0$ 时，依次尝试一个中间构型 w，将问题分成从 u 到 w、从 w 到 v 两段，每段的长度上限都是 2^{k-1}。因此，两次递归的参数分别为 $(u, w, k-1)$ 和 $(w, v, k-1)$。先检查前半段，如果它不存在，就换下一个 w。如果前半段存在，再检查后半段。只要找到一个 w 使两段都存在，就回答存在，所有 w 都失败后才回答不存在。枚举时，按顺序生成 b 位二进制串，跳过不表示合法构型的编码即可。最后，依次枚举所有接受构型 v，从起始构型出发，以 $k = b$ 调用上述递归。找到一个可达的接受构型就接受输入，全部不可达则拒绝。

计算上述算法使用的空间。每层需要保存两个端点 u, v、当前尝试的中间构型 w、参数 k 和递归返回的真假结果。三个构型各占 b 位，k 至多为 b，所以每层合计只需 $O(b)$ 位。最底层检查一条转移规则时，也只需 $O(b)$ 空间。递归前半段检查结束后，只保留它的真假结果，其余空间都可以释放，交给后半段使用。因此，计算空间时只需累加当前尚未结束的各层调用，至多有 $b+1$ 层，故总空间为 $O(b^2) = O(s(N)^2)$。 □

如果原来的空间界 $s(N)$ 是多项式，Savitch 定理给出的 $s(N)^2$ 仍是多项式，所以 $\mathsf{NPSPACE} = \mathsf{PSPACE}$。此外，将确定性判定图灵机的接受与拒绝互换，就能判定补语言，使用的空间不变。因此 $\text{co-}\mathsf{PSPACE} = \mathsf{PSPACE}$。

现在可以把时间类与空间类联系起来。一个只运行多项式步的计算分支，最多访问多项式多个纸带格，所以 $\mathsf{NP} \subseteq \mathsf{NPSPACE} = \mathsf{PSPACE}$。反之，使用多项式空间的确定性图灵机只有 $2^{N^{O(1)}}$ 个

可能的构型。如果某次计算重复了一个构型，以后的转移也会重复，图灵机会一直循环。判定图灵机必须停机，故不会重复构型，运行时间也就不超过这个指数上界。这证明了 PSPACE $\subseteq$ EXP。综上，

$$\mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}.$$

目前还不知道其中任意一对相邻的类是否相等。后面的时间、空间层次定理与间隙定理部分将证明 P $\subsetneq$ EXP，因此这串包含关系中至少有一处是严格包含。

上面比较时间与空间时，用到了一个直接的估计：每步只能访问常数个纸带格子，所以运行 $t(n)$ 步至多使用 $O(t(n))$ 空间。这里假设输入长度为 n 且 $t(n) \geqslant n$，因而输入占用的格子也包含在这个上界内。如果允许重新组织计算，花更多时间来节省空间，能否用远少于 $t(n)$ 的空间实现相同计算？对于确定性多带图灵机，已有结果能将模拟所需的空间降到 $O(t(n)/\log t(n))$。Williams 在 2025 年将这一保持近五十年的上界进一步改进为 $O\left(\sqrt{t(n)\log t(n)}\right)$。也就是说，任何在 $t(n)$ 步内完成的计算，都可以由另一台确定性图灵机用上述空间完成，并在每个输入上给出相同结果，不过运行时间可能更长。证明的方法是将原计算转化为一系列树求值问题，再利用节省空间的求值算法完成模拟。[3]

全量化布尔公式与博弈

下面看一个可以用多项式空间求解的具体问题。SAT 允许我们选择全部变量的值，只要找到一组使公式为真的赋值即可。如果部分变量由对手选择，我们要根据他的选择决定其他变量，问题就不同了。为了表示双方按顺序作出的选择，可以给布尔公式加上存在量词和全称量词。例如 $\forall x \exists y\,(x \leftrightarrow y)$ 为真，因为看到 x 后总能选择 $y = x$。如果交换顺序，写成 $\exists y \forall x\,(x \leftrightarrow y)$，则无论先选哪个 y，随后取 $x = 1 - y$ 都会使公式为假。量词顺序决定了作出选择时能够使用哪些信息。

考虑形如 $Q_1 x_1 \cdots Q_n x_n\, F(x_1, \ldots, x_n)$ 的公式，其中每个 Q_i 都是 $\exists$ 或 $\forall$，每个变量都取 0 或 1。公式中的全部变量都有对应的量词，因此称为**全量化布尔公式（fully quantified Boolean formula）**。TQBF 要求判断一个这样的公式是否为真，也用来表示由这些真公式组成的语言。

可以把求值过程理解为两个人轮流作选择的博弈。双方按照量词的先后顺序给变量赋值，一方负责所有 $\exists$ 变量，希望最后的 F 为真，另一方负责所有 $\forall$ 变量，希望 F 为假。前者称为存在方，后者称为全称方。公式为真，就表示存在方无论面对对手怎样的选择，都能让最后的 F 为真。

判断整个公式的真值，可以从第一个变量开始递归。例如，对 $\forall x \exists y\,(x \leftrightarrow y)$，先固定 $x = 0$，求出 $\exists y\,(0 \leftrightarrow y)$ 的真值，再固定 $x = 1$，求出 $\exists y\,(1 \leftrightarrow y)$ 的真值。两个结果都为真，而最外层是全称量词，所以原公式为真。如果最外层换成存在量词，就只需两个结果中有一个为真。

一般地，已经固定 $x_1, \ldots, x_{i-1}$ 后，分别令 $x_i = 0$ 和 $x_i = 1$，递归判断剩余的量词公式。若 $Q_i = \exists$，就对两次结果取或，若 $Q_i = \forall$，就取与。一直递归到全部变量都有取值时，直接计算不含量词的 F，再逐层返回结果，就能求出整个公式的真值。这相当于深度优先遍历赋值树。求完 $x_i = 0$ 对应的子树后，只需记住返回的一个真假值，随后就可以将 x_i 改为 1，继续计算另一棵子树。两次计算共用同一个赋值数组，前一次递归释放的空间也可以交给后一次使用。因此，算法只需保存当前的 n 个变量值，以及至多 $n + 1$ 层尚未结束的递归调用。每层记录当前变量的下标、正在尝试哪个取值和第一次递归的结果，所需空间为 $O(\log n)$ 位。再加上计算 F 真值的空间，总空间仍为输入公式长度的多项式。这就证明了 TQBF $\in$ PSPACE。

> **定理 10.14.** TQBF 是 PSPACE 完全的。

[3] R. Ryan Williams，*Simulating Time With Square-Root Space*，2025。

证明. 上面已经证明 TQBF 属于 PSPACE。为了证明它是 PSPACE 困难的，我们要将任意 PSPACE 问题的输入 x 转成一个全量化布尔公式，使公式为真当且仅当原问题的答案为是，而且转换只需多项式时间。取任意 $L \in$ PSPACE，固定一台判定它的图灵机。按照 Savitch 定理证明中的编码方法，对输入 x，每个构型可以用 $b = |x|^{O(1)}$ 位表示。原图灵机接受 x，当且仅当构型图中存在从起始构型到接受构型的路径。下面把这个可达性条件写成公式。

先处理至多走一步的情形。用 U, V 分别表示两组 b 个布尔变量，每组变量的取值表示一个构型的编码。构造公式 $E(U, V)$，要求这两个编码都合法，并且 $U = V$ 或从 U 能一步转移到 V。构造方法与 Cook–Levin 定理中检查相邻两行相同。先检查编码中的状态、磁头位置和纸带符号，再检查是否有一条规则能使前一个构型变成后一个构型。被访问的格子按规则改变，其余格子保持不变。图灵机的规则数固定，即使枚举所有磁头位置并逐格比较，得到的公式长度也只是 b 的多项式。

再考虑多步路径。用公式 $R_k(U, V)$ 表示从 U 到 V 是否存在长度至多为 2^k 的路径，因此 $R_0(U, V) = E(U, V)$。对于 $k > 0$，沿用前面的分治思路，选择一个中间构型 W，要求前后两段的长度都至多为 2^{k-1}，就得到 $\exists W\,[R_{k-1}(U, W) \wedge R_{k-1}(W, V)]$，这里 $\exists W$ 表示对 W 的全部 b 个比特使用存在量词。但这个写法每次都要复制两份 R_{k-1}，展开 b 层就会得到指数长度的公式，无法在多项式时间内输出。我们需要让两段路径共用一份公式。为此，再引入两组 b 位变量 A, B，用它们表示要检查的两个端点。要求对所有 A, B，只要它们等于 (U, W) 或 (W, V)，就必须满足 $R_{k-1}(A, B)$。这样，两段路径都受到检查，而公式中只需写一次 R_{k-1}。具体写成

$$R_k(U, V) = \exists W\, \forall A\, \forall B\ \Big(\big[(A = U \wedge B = W) \vee (A = W \wedge B = V)\big] \Rightarrow R_{k-1}(A, B)\Big).$$

固定 W 后，代入 $A = U, B = W$，上述公式便要求前半段可达。代入 $A = W, B = V$，便要求后半段可达。其余取值使蕴含式的前件为假，公式自动成立。因此，这条全称条件恰好要求两段路径都存在。再由存在量词选择 W，就得到长度至多为 2^k 的可达性条件。

上述公式每展开一层，只增加对几个 b 位编码的比较和 $3b$ 个量词，额外需要 $O(b)$ 个符号。展开 b 层后，公式长度为 $|E| + O(b^2)$，仍是输入长度的多项式。构型图至多有 2^b 个顶点，因此检查长度至多为 2^b 的路径就足够了。将起始构型的编码作为常量 $start$，再用 Accept(V) 表示 V 是合法的接受构型，最终输出 $\exists V\ \big(\text{Accept}(V) \wedge R_b(start, V)\big)$。这个公式要求存在一个从起点可达的接受构型，所以它为真当且仅当原图灵机接受 x，即 $x \in L$。检查接受构型的公式也只有多项式长度。最后，将量词移到最前面，即可整理成 TQBF 的输入形式。整个整理过程所需时间和最终公式长度都为多项式。因此 $L \leqslant_P$ TQBF，定理得证。 □

多项式层级

前面证明了 TQBF 是 PSPACE 完全问题，但如果限制公式中量词的排列方式，就会得到我们已经讨论过的问题。例如，所有量词都是 $\exists$ 时，问题就是判断是否存在一组满足赋值，也就是 SAT。所有量词都是 $\forall$ 时，问题就是判断每组赋值是否都使公式为真，也就是永真式问题。这两种情况分别属于 NP 和 co-NP，它们的共同特点是量词完全不交替，而一般的 TQBF 没有这个限制。因此，我们可以进一步研究，只允许量词交替一次、两次，或者某个固定次数时，会得到哪些复杂度类。下面介绍的多项式层级，就是沿着这个思路，按照量词交替的多少对问题进行分类。

为了按这种量词结构分类，我们把连续的同类量词合为一个**量词块（quantifier block）**。例如，$\exists u \exists v \forall w$ 的前两个量词可以合写为 $\exists(u, v)$，因此它共有两个量词块。每个量词块中的变量取值，可以一起写成一个二进制串。我们希望延续前面用证书定义 NP 的方法，因此要求这些串的长度不超过输

入长度的某个多项式，给定它们以后，也能在多项式时间内检查条件是否成立。如果只有存在量词，问题就是能否找到一个串，使检查通过，这正是 NP 的证书定义。如果只有全称量词，则补问题属于 NP，原问题便属于 co-NP。现在固定正整数 k，允许使用 k 个交替的量词块。如果从存在量词开始，就得到语言类 Σ_k^{P}。具体地，$L \in \Sigma_k^{\mathsf{P}}$ 是指存在多项式时间检查算法 V，使

$$x \in L \iff \exists u_1 \forall u_2 \cdots Q_k u_k : V(x, u_1, \ldots, u_k) = 1.$$

这里每个 u_i 都遍历长度不超过某个多项式 $p(|x|)$ 的二进制串。如果从全称量词开始，其余要求相同，就得到 Π_k^{P}。因此，$\Sigma_1^{\mathsf{P}} = \mathsf{NP}$，$\Pi_1^{\mathsf{P}} = \text{co-NP}$。没有量词时，直接运行 V 即可，所以约定 $\Sigma_0^{\mathsf{P}} = \Pi_0^{\mathsf{P}} = \mathsf{P}$。对成员条件整体取否定，会交换全部量词，并把检查结果取反，故 $\Pi_k^{\mathsf{P}} = \text{co-}\Sigma_k^{\mathsf{P}}$。

如果在最里面增加一个不参与检查的新变量，再给它加上与前一块不同的量词，原条件的真假不变，量词块却多了一个。因此 $\Sigma_k^{\mathsf{P}} \subseteq \Sigma_{k+1}^{\mathsf{P}}$。把所有固定层数合在一起，就得到**多项式层级（polynomial hierarchy）**

$$\mathsf{PH} = \bigcup_{k \geqslant 0} \Sigma_k^{\mathsf{P}}.$$

从全称量词开始的条件也包括在内，因为可以在最外面加一个不参与检查的存在量词。按照前面求解 TQBF 的深度优先搜索方法，就能用多项式空间完成判断，所以 $\mathsf{PH} \subseteq \mathsf{PSPACE}$。目前尚不知道 PH 与 PSPACE 是否相等。对于多项式层级，我们已经知道 $\Sigma_k^{\mathsf{P}} \subseteq \Sigma_{k+1}^{\mathsf{P}}$，但目前还不知道这一包含是否严格。即使是第 0 层与第 1 层是否相等，也就是 P 与 NP 是否相等，仍未解决。

如果 $\mathsf{P} = \mathsf{NP}$，那么整个多项式层级都等于 P。证明时从最内层开始，将外层变量的取值视为输入。起初只需运行多项式时间算法 V。假设已经能在多项式时间内判断条件 R，接着处理它外面的一个量词块。对于 $\exists u\, R$，给出 u 就能检查 R 是否成立，所以这个问题属于 NP，由假设也属于 P。对于 $\forall u\, R$，先判断是否存在使 R 不成立的 u。给出这样的 u 后同样能快速检查，所以寻找反例的判定问题也属于 $\mathsf{NP} = \mathsf{P}$，将答案取反即可判断 $\forall u\, R$。因此每处理一块，所得问题仍属于 P。块数固定，整个判断也就只需多项式时间。

如果只假设 $\mathsf{NP} = \text{co-NP}$，也能证明整个多项式层级等于 NP。从 V 开始归纳，假设条件 R 属于 NP。对于 $\exists u\, R$，同时给出 u 和证明 R 成立的证书，就能完成验证，所以仍属于 NP。对于 $\forall u\, R$，它不成立当且仅当 $\exists u\, \neg R$。由 $\mathsf{NP} = \text{co-NP}$，判断 $\neg R$ 也属于 NP，因此 $\exists u\, \neg R$ 属于 NP。它的补问题 $\forall u\, R$ 于是属于 $\text{co-NP} = \mathsf{NP}$。这样，无论增加哪种量词块，所得问题都留在 NP 中。如果某个固定层及其所有更高层都相等，就称多项式层级发生了**坍缩（collapse）**。上述两个假设分别使它坍缩到第 0 层和第 1 层。

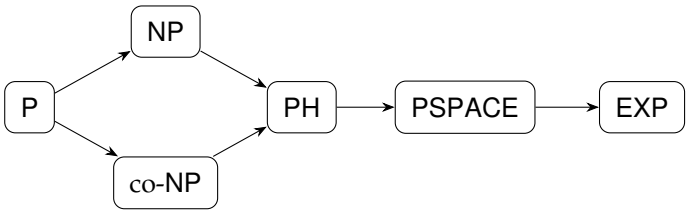


图 10.8 复杂度类间的包含关系，箭头从较小的类指向包含它的类。$\mathsf{P} \subsetneq \mathsf{EXP}$，相邻两类是否相等尚未确定。

10.6.2 可计算性与不完备性

前面讨论的问题都有求解算法，我们关心的是需要多少时间和空间。现在进一步问，是否每个问题都能用算法解决？为此，我们先介绍怎样给程序和公式编码，将它们也作为算法可以处理的对象。

Gödel 编码与通用图灵机

要让一个程序分析另一个程序，首先得把后者的描述交给它作为输入。对于图灵机，这很容易做到，因为它的状态、纸带符号和转移规则都是有限的。给状态和符号编号，再把规则逐条写下来，就能用一个有限字符串完整描述一台图灵机 M，记为 $\langle M\rangle$。只要约定好格式，读入这个字符串就能恢复 M 的规则表。公式和证明同样由有限个符号组成，也可以这样表示。

还可以进一步用一个自然数保存整个字符串，这称为 **Gödel 编码（Gödel numbering）**。这样，讨论一条公式或一份证明时，就可以用它的编号来指代它。一种具体做法是给不同字符分配不同的正整数编号 $f(a)$，再把第 i 个字符的编号作为第 i 个素数的指数。记依次排列的素数为 $p_1, p_2, \ldots$，则字符串的编码为

$$g(x_1 \cdots x_n) = p_1^{f(x_1)} \cdots p_n^{f(x_n)}, \qquad g(\varepsilon) = 1.$$

例如，取 $f(a) = 1, f(b) = 2$，字符串 ab 就编码为 $2^1 \cdot 3^2 = 18$，ba 则编码为 $2^2 \cdot 3^1 = 12$。反过来，将 18 分解成 $2^1 \cdot 3^2$，读出指数 $1, 2$，就恢复了 ab。唯一分解定理保证这种恢复只有一个结果，因此不同字符串不会得到相同的编码。依次检查字符串或自然数，并保留合法的图灵机描述，还能将全部图灵机列成 $M_1, M_2, \ldots$。

有了图灵机的描述，我们就可以设计一台固定的图灵机 U，让它读入 $\langle M\rangle$ 和字符串 x，模拟 M 在输入 x 上的运行。这样的 U 称为**通用图灵机（universal Turing machine）**。它先读出 M 的规则表，再用自己的纸带保存 M 的状态、磁头位置和纸带内容。每模拟一步，U 就查找当前适用的规则进行更新。如果 $M(x)$ 停机，U 也停止并给出相同结果。如果 $M(x)$ 一直运行，U 就一直模拟。因此，同一台 U 只需换一个输入描述，就能模拟不同的图灵机，如同同一个解释器可以执行不同的程序。

停机问题

通用图灵机能够模拟任意程序，但如果被模拟的程序始终不停止，模拟也就一直进行下去。能不能在模拟之前先判断，一个程序在给定输入上究竟会不会停机？这就是**停机问题（halting problem）**。它对应的语言为

$$\mathrm{HALT} = \{(\langle M\rangle, x) : M \text{ 是图灵机，且在输入} x \text{ 上停机}\}.$$

如果 $M(x)$ 停机，这个字符串就属于 HALT，否则就不属于。判定停机问题，就是要求算法对每个这样的输入都能在有限时间内回答它是否属于 HALT。下面证明，这样的算法不存在。

> **定理 10.15.** 语言 HALT 不可判定，即不存在一个对任意 $(\langle M\rangle, x)$ 都停机并正确判断 $M(x)$ 是否停机的算法。

证明. 假设存在判定算法 H，输入图灵机的描述及其输入字符串后，H 总会停止，并用 1 表示会停机，0 表示不会停机。我们利用 H 构造程序 D。D 收到字符串 z 后，先问 H：如果把 z 既当作程序描述，又当作这个程序的输入，程序会不会停止？然后故意作出相反的行为，H 回答会停止时就无限循环，回答不会停止时就立即返回。代码如下：

> **算法 10.1　在假设的停机判定器下构造反例程序**
>
> **输入：** 程序描述字符串 z
>
> 1: **if** $H(z, z) = 1$ **then** 永远循环
> 2: **else return** 0

如果 z 不是合法的程序描述，就将它解释为一个立即停机的程序，使 D 对所有字符串都有定义。

D 本身也是一个有限程序，设它的描述为 $d = \langle D \rangle$。现在让 D 处理自己的描述 d。如果 $H(d,d) = 1$，判定器认为 $D(d)$ 会停机，但 D 恰好在这种情况下进入无限循环。如果 $H(d,d) = 0$，判定器认为它不会停机，但 D 又会立即返回。两种回答都会出错，所以这样的判定器 H 不存在。 □

注意，停机问题可以识别：用通用图灵机模拟输入给出的计算，模拟到停机时就接受，计算一直继续时也跟着继续模拟。因此，停机问题给出了一个可以识别、却无法判定的语言。停机问题还给出了一个不属于 NP 的例子，因为停机问题甚至不可判定。另一方面，停机问题是 NP 困难的。给定 SAT 公式 F，构造一台忽略自身输入的图灵机 M_F，让它枚举 F 的所有赋值。如果找到满足赋值，就停机，如果全部赋值都失败，就进入无限循环。于是 $F \in \mathrm{SAT} \iff M_F$ 在空输入上停机。把 F 写入这台图灵机的描述，并附上固定的枚举过程，就能在多项式时间内构造 $(\langle M_F \rangle, \varepsilon)$，从而得到 SAT 到停机问题的归约。停机问题因此也说明，NP 困难问题可以超出 NP，甚至无法判定。

刚才构造 D 时，关键是让它在自己的描述上采取与预测相反的行动，从而使预测必定出错，这就是对角线法。用它还能证明，不存在一个算法，能够完整枚举所有从自然数到自然数的可计算全函数。这里的全函数是指，对每个自然数输入都有定义的函数，计算它的算法在每个输入上都会停机。假设已有这样的列表 $f_1, f_2, \ldots$，并且有算法在任意输入 (i,x) 上计算 $g(i,x) = f_i(x)$。由于每个 f_i 都处处有定义，$h(i) = g(i,i) + 1$ 也处处有定义且可计算。但是对每个 i，都有 $h(i) \neq f_i(i)$，所以 h 与列表中的每个函数都不同，产生矛盾。这一例子也更直观地体现了“对角线”的含义。

对角线引理与 Tarski 定理

停机问题的证明让程序读入了自己的描述。接下来，我们希望在算术中表达类似的自指，例如构造一条语句，说它自己无法被证明。为此，先说明算术语句怎样写出，以及证明是从什么出发的。

算术中用 $0, 1$、变量以及加法、乘法等符号写出表示数值的表达式，例如 $(v+1)\cdot v$，称为**项（term）**。把两个项用等号连接起来，就能陈述一个条件，例如 $v + 1 = 1$。再使用否定、合取、量词等逻辑符号，就能写出更复杂的**公式（formula）**。不含自由变量的公式称为**语句（sentence）**，也称为句子。

要把某个具体的自然数代入变量，就需要在公式中写出表示它的项。例如，把 $v + 0 = v$ 中的 v 换成 $((0+1)+1)+1$，就是代入自然数 3。一般地，从 0 开始加 n 次 1，便得到表示 n 的项，记为 $\overline{n}$，称为**数码（numeral）**。这样，只用固定的符号 $0, 1, +$ 就能写出任意自然数。平时写的 3、17 可以看作这些项的简写。后面编号为 n 的公式要谈论自己时，就把数码 $\overline{n}$ 代入相应的变量。

写出语句以后，还要说明如何证明它。我们选定一套算术公理，并使用固定的逻辑推理规则，由这些公理及其能够推出的结论组成的体系称为**理论（theory）**，记为 T。例如，可以把 $\forall v\,(v + 0 = v)$ 选作一条公理，再由它推出 $\overline{3} + 0 = \overline{3}$。记号 $T \vdash A$ 表示能从 T 的公理出发，经过有限步推理证明语句 A。另外，$\mathbb{N} \models A$ 表示按通常的自然数及其运算解释 A 时，它为真。

有了这些约定，就可以利用 Gödel 编码把关于公式的陈述写成算术公式。例如，若 $B(v)$ 表达编号为 v 的公式具有某种性质，那么 $B(\overline{g(A)})$ 就是在陈述公式 A 具有这种性质。我们希望进一步找到一个句子 A，让它自己恰好表达 $B(\overline{g(A)})$。下面的对角线引理将说明如何做到这一点。

构造自指句子的关键，是把一条公式自己的编号代入它。设 $A(v)$ 只有一个自由变量，编号为 $e = g(A)$。把 $\overline{e}$ 代入 v，就得到句子 $A(\overline{e})$。定义 $d(e)$ 为这个新句子的编号，因而 d 完成的是“按编号找到公式、代入它自己的编号、为结果重新编码”这三个步骤。如果 e 不是合法的一元公式编号，就令 $d(e) = 0$。这些步骤都能由算法完成。接下来要把这个计算过程写进算术公式。我们用 $D(v,w)$

表达“以 v 为输入执行上述代入和编码操作，得到的结果是 w”。后面的证明需要在理论 T 中使用这个计算结果，因此我们要求，对每个给定的输入 e，都能从 T 的公理证明结果恰好是 $d(e)$。用符号表示，就是 $T \vdash \forall w\big(D(\overline{e}, w) \leftrightarrow w = \overline{d(e)}\big)$。这个等价式说的是，把输入固定为 e 后，$D(\overline{e}, w)$ 成立的唯一可能就是 $w = d(e)$。这样，在后面的推理中，只要知道 $D(\overline{e}, w)$，就能把 w 换成已经算出的结果 $\overline{d(e)}$。这里把这一性质作为对 T 的要求，Peano 算术等常用的算术理论能够满足它。

引理 10.16 (对角线引理)**.** 在上述条件下，对任意只有一个自由变量的公式 $B(v)$，存在句子 A，使得

$$T \vdash A \leftrightarrow B(\overline{g(A)}).$$

证明. 我们要构造的句子 A，应当恰好表达“编号 $g(A)$ 具有性质 B”。困难在于，写出 A 以前还不知道它的编号，所以先借助代入操作 d，构造带有自由变量的公式 $C(v) = \exists w(D(v, w) \wedge B(w))$。它表示，编号 v 对应的公式代入自己的编号后，所得句子的编号具有性质 B。现在 C 已经写出，可以求出其编号 $e = g(C)$，再令 $A = C(\overline{e})$。按 d 的定义，$d(e)$ 正是 A 的编号。于是

$$A \leftrightarrow \exists w(D(\overline{e}, w) \wedge B(w)) \leftrightarrow B(\overline{d(e)}) \leftrightarrow B(\overline{g(A)}).$$

第一步展开 A 和 C 的定义。第二步利用 T 能证明 $D(\overline{e}, w)$ 当且仅当 $w = \overline{d(e)}$。最后代入 $d(e) = g(A)$，就得到所需等价式，而且这些步骤都能在 T 内证明。 □

定理 10.17 (Tarski 不可定义性，算术语义版本)**.** 设 $d(e)$ 是将编号为 e 的公式代入自身编号后所得句子的编号，并假设存在算术公式 $D(v, w)$，使它在自然数中成立当且仅当 $w = d(v)$。则不存在算术公式 $\mathrm{True}(v)$，对每个算术句子 A 都满足

$$\mathbb{N} \models \mathrm{True}(\overline{g(A)}) \leftrightarrow A.$$

证明. 假设这样的 True 存在。在对角线引理的构造中取 $B(v) = \neg\mathrm{True}(v)$，并按通常的自然数解释各公式，就得到一个句子 S，它表达自己不为真，即 $\mathbb{N} \models S \leftrightarrow \neg\mathrm{True}(\overline{g(S)})$。如果 True 确实能判断所有算术句子的真值，那么 $\mathrm{True}(\overline{g(S)})$ 就应当与 S 同真同假。代入上式便得到 $S \leftrightarrow \neg S$，矛盾。 □

Tarski 定理说明，无法用一条算术公式正确表达所有算术句子的真值。接下来换一个问题，给定一套算术公理，能否对每个句子都证明它或它的否定?

Gödel 不完备定理

我们通常从一套公理出发，按照推理规则证明结论。如果对每个句子 A，都能证明 A 或者证明 $\neg A$，就称这套理论是完备的。Gödel–Rosser 定理将说明，常用的算术理论只要没有矛盾，就做不到这一点。具体地，设理论 T 的公理能够由算法依次列出，T 不会同时证明一个句子及其否定，并且包含足够的初等算术，能够表达和验证下面用到的编码、代入及有限计算。Peano 算术及其一致扩张满足这里所需的算术条件。

为了在 T 中讨论证明，我们也把每份证明编码为自然数，并采用能够由算法在有限步内检查的证明格式。用 $\mathrm{Proof}_T(p, a)$ 表示“编号为 p 的对象是一份合法证明，其结论的编号为 a”。后面的论证需要 T 能确认这些具体的检查结果。例如，给定一份证明和一个句子，T 能证明它确实是这个句

子的证明，或者证明它不是。如果要检查编号不超过某个固定数的全部对象，就逐个检查，再把结果合在一起。这里将这些能力作为对 T 的算术条件。

定理 10.18 (Gödel–Rosser 不完备性)**.** 满足这些条件的 T 存在句子 R，使得 $T \nvdash R$ 且 $T \nvdash \neg R$。

证明. 我们希望构造一个句子，使证明它和证明它的否定都会导致矛盾。为此，利用对角线引理构造句子 R，让它表达“如果有一份我的证明，就有一份编号不更大的、证明我不成立的证明”。这就是 Rosser 句子。正式写成

$$R \leftrightarrow \forall p \left(\mathrm{Proof}_T(p, \overline{g(R)}) \Rightarrow \exists q \leqslant p \, \mathrm{Proof}_T(q, \overline{g(\neg R)}) \right).$$

这里还需要根据 R 的编号求出 $\neg R$ 的编号。具体做法是先解码得到句子 R，写出它的否定 $\neg R$，再按 Gödel 编码规则计算新句子的编号。在对 T 的算术条件下，这一计算过程也能用算术公式表达，因此可以用于构造上面的 Rosser 句子。

先假设 T 能证明 R，取一份证明的编号 p_0。由于 T 一致，它就没有 $\neg R$ 的证明。因此，把编号从 0 到 p_0 逐个检查，都会发现它们不是 $\neg R$ 的证明。这些检查只有有限步，T 能证明每次检查的结果，也能证明 p_0 确实编码了 R 的证明。于是，在上面的全称式中取 $p = p_0$，蕴含式的前件为真，后件为假。T 因此又能推出 $\neg R$，与一致性矛盾。所以 T 不能证明 R。

再假设 T 能证明 $\neg R$，取一份证明的编号 q_0。刚才已经证明，R 没有证明，所以编号小于 q_0 的每个对象都不是 R 的证明，T 可以通过有限次检查证明这一点。再看定义 R 的全称式。对于 $p < q_0$，刚才的有限检查已经说明，p 不是 R 的证明编号，因而蕴含式的前件为假。对于 $p \geqslant q_0$，直接取 $q = q_0$，就有一份编号不超过 p 的 $\neg R$ 的证明，因而后件成立。这两个范围包含所有 p，并且两种情形都能在 T 内证明，所以 T 能证明整个全称式，也就能证明 R。这与假设中已有的 $\neg R$ 的证明矛盾。因此，T 也不能证明 $\neg R$。 □

Hilbert 曾希望通过形式化方法为数学建立可靠的基础。Gödel–Rosser 定理说明，即使公理和推理规则已经明确给出，只要这样的算术理论没有矛盾，仍会存在既无法证明也无法否定的句子。

10.6.3 复杂度层次与证明方法

时间可构造性与线性加速

前面证明了有些问题根本没有判定算法。下面回到可以判定的问题，研究给算法更多时间或空间，是否一定能解决更多问题。为了证明增加时间确实能解决更多问题，后面会用对角线法构造一个新算法，让它模拟时间受限的算法，再给出相反的答案。模拟必须在规定的步数内结束，所以先要解决一个具体问题：怎样让图灵机在运行指定步数后停止？

设 $t : \mathbb{N} \to \mathbb{N}$ 且 $t(N) \geqslant N$。如果给定由 N 个 1 组成的输入，图灵机能够在 $O(t(N))$ 时间内写出 $t(N)$ 个 1，就称 t 为**时间可构造（time-constructible）**函数。这串 1 可以放在一条单独的纸带上作为计时器，每执行一步就划去一个，全部划去时停止。准备和使用这个计时器都只需 $O(t(N))$ 时间，因而不会超出要证明的时间界。如果只要求 t 能够计算出来，计算 $t(N)$ 本身可能已经超过这个时间界，这正是需要“时间可构造”条件的原因。

比较不同时间界时，先看常数倍的差异。在多带图灵机模型中，通过扩大字母表和状态集，可以让一台新的图灵机一次模拟原来的若干步，从而缩短模拟所需的时间。具体地，给定运行时间为

$T(N)$ 的图灵机和固定常数 $\varepsilon > 0$，可以在 $\varepsilon T(N) + O(N)$ 时间内完成相同计算，这称为**线性加速（linear speedup）**。具体做法是把旧纸带上连续的 b 个格子合成一个新符号，并在有限状态中记录旧磁头在块内的位置。模拟时，读取磁头所在块及左右相邻块。旧图灵机在接下来的 b 步里最多移动 b 格，所以需要的信息都在这三个块中。因为 b 和旧图灵机都是固定的，可以事先把这段计算的所有可能情况写入新图灵机的有限规则表。新图灵机查表后改写相应块并更新位置，就完成了旧图灵机的 b 步。每批只需常数次块操作，因此取足够大的 b，就能使模拟时间的系数小于给定的 ε。最初将输入整理成块需要 $O(N)$ 时间，最后不足一批的计算只增加常数时间，合起来便得到所述上界。这说明，时间的常数系数可以通过改变图灵机的实现方式来缩小。下面研究时间界增长速度的差异，例如从多项式时间增加到指数时间，能否解决更多问题。

时间、空间层次定理与间隙定理

我们先证明 $\mathsf{P} \subsetneq \mathsf{EXP}$，也就是有些问题虽然能在指数时间内解决，却没有多项式时间算法。思路与停机问题的证明相似，构造一个新算法，让它与每个多项式时间算法至少在一个输入上给出不同答案。为此，枚举由图灵机 M 和正整数 k 组成的二元组 (M, k)，让 M 在长度为 N 的输入上至多运行 $(N+1)^k + k$ 步，超时就拒绝。只要选取足够大的固定 k，任何多项式时间判定算法都可以用这种形式表示。同一二元组的描述还可以添加任意长的注释，解释器读入后忽略注释即可。因此，同一台图灵机有任意长的描述，后面的证明可以选择其中一个足够长的描述作为输入。

现在构造算法 D。输入字符串 x 描述了二元组 (M, k)，D 就把这个字符串也作为 M 的输入，按前面规定的时间限制模拟 M，然后给出相反的答案。为保证 D 本身在指数时间内停机，设 $N = |x|$，只允许整个模拟过程执行 2^{N^2} 步。在此之前得到答案就取反，模拟超时或输入不是合法描述则回答 0。准备计时器和完成模拟共需 $2^{O(N^2)}$ 时间，所以 D 判定的语言 L_D 属于 EXP。

假设 $L_D \in \mathsf{P}$，那么就有一台多项式时间图灵机 M，对每个输入都给出与 D 相同的答案。选定足够大的 k，使 M 总能在 $(N+1)^k + k$ 步内完成计算。现在把 (M, k) 的描述写成字符串 x，交给 D。按照 D 的定义，它会运行模拟过程，求出 M 对这个同一字符串 x 的回答，再将答案取反。关键是保证 D 能在模拟超时之前得到 $M(x)$ 的答案。由于 M 只运行多项式步，D 逐步模拟它也只需 $N^{O(1)}$ 时间，其中 $N = |x|$。给描述 x 添加足够长的注释，就能使 N 足够大，让这段模拟在 2^{N^2} 步内完成。于是 $D(x)$ 与 $M(x)$ 必定相反，与两者判定同一个语言矛盾。因此 $L_D \notin \mathsf{P}$，结合前面已知的 $L_D \in \mathsf{EXP}$，便得到 $\mathsf{P} \subsetneq \mathsf{EXP}$。

上面的证明区分了多项式时间与指数时间。更一般地，**确定性时间层次定理（deterministic time hierarchy theorem）**比较两个给定的时间界。设 f, g 都是时间可构造函数，满足 $f(N), g(N) \geqslant N$ 且 $f(N)\log(f(N)+1) = o(g(N))$，则 $\mathsf{DTIME}(f(N)) \subsetneq \mathsf{DTIME}(g(N))$。证明思路与前面相同，构造一个新算法，模拟输入所描述的图灵机，再将它的答案取反。模拟一个运行 $O(f(N))$ 步的固定图灵机需要 $O(f(N)\log(f(N)+1))$ 时间，因此定理的条件保证，在足够长的描述输入上，$O(g(N))$ 时间足以完成模拟并取反。新算法由此与每个 $O(f(N))$ 时间算法至少在一个输入上给出不同答案，证明增加时间后确实能判定更多语言。

空间也有对应的层次定理。设 f, g 都是空间可构造函数，满足 $f(N), g(N) \geqslant \log(N+2)$ 且 $f = o(g)$，则 $\mathsf{DSPACE}(f(N)) \subsetneq \mathsf{DSPACE}(g(N))$。空间层次定理也用模拟后取反的方法证明。模拟一台固定图灵机只需将它使用的空间增加常数倍，所以 $f = o(g)$ 就足以让新算法模拟每个 $O(f(N))$ 空间算法，并在一个足够长的描述输入上给出相反答案。为使新算法总能停机，还要限制模拟步数。使用 $O(g(N))$ 空间的计算至多有 $2^{O(g(N))}$ 个构型，超过这个步数仍未停机就必然陷入循环，而记录步数只需 $O(g(N))$ 空间。

非确定性时间也有层次定理。若 f, g 都是时间可构造函数，满足 $f(N), g(N) \geqslant N$ 且 $f(N+1) = o(g(N))$，则 $\mathsf{NTIME}(f(N)) \subsetneq \mathsf{NTIME}(g(N))$。非确定性图灵机只要有一条接受分支就接受，所以交换各分支的接受和拒绝，并不能保证整体答案相反。证明这个结果需要进一步安排模拟的顺序，所用方法称为**延迟对角化（delayed diagonalization）**，详见参考资料中的复杂度教材。

时间层次定理要求时间界可构造。如果只要求时间界能够算出，增加时间就未必能解决更多问题，这就是下面的间隙定理所说明的。

> **定理 10.19** (Borodin–Trakhtenbrot 间隙定理)**.** 设函数 $g : \mathbb{N} \to \mathbb{N}$ 在每个输入上都有定义且可计算，并满足 $g(u) \geqslant u$。则存在同样处处有定义且可计算的函数 t，它单调不减，满足 $t(N) \geqslant N$，并且
>
> $$\mathsf{DTIME}(t(N)) = \mathsf{DTIME}(g(t(N))).$$

Ladner 定理

如果 $\mathsf{P} \neq \mathsf{NP}$，是否每个没有多项式时间算法的 NP 问题都是 NP 完全问题？Ladner 定理给出了否定回答。属于 NP、不属于 P，又不是 NP 完全的问题，称为 NP **中间问题（NP-intermediate problem）**。

> **定理 10.20** (Ladner)**.** 若 $\mathsf{P} \neq \mathsf{NP}$，则存在 NP 中间问题。

一种证明思路是在 SAT 公式后补上无关字符，仍然只问原公式是否可满足。补上的字符越多，输入就越长，原来的求解时间相对于新输入长度便越小。例如，补上指数多个字符后，枚举赋值已经只需新输入长度的多项式时间。证明通过控制填充长度的增长，让新问题保留足够的困难性，同时逐一排除多项式时间判定算法和从 SAT 到它的多项式时间归约，最终得到所需的中间问题。当然，定理虽然保证了这类问题的存在，但并未确定某个常见问题就是 NP 中间问题。

谕示图灵机与相对化

前面的对角线法能证明 $\mathsf{P} \subsetneq \mathsf{EXP}$，能否也用来区分 P 与 NP？理解这类证明方法的局限，可以借助一个思想实验。假设图灵机能够调用语言 A 的理想判定器，写出查询字符串后，只需一步就能获知它是否属于 A。这个判定器称为**谕示（oracle）**，使用它的图灵机称为**谕示图灵机（oracle Turing machine）**。写出查询字符串仍计入运行时间。

在谕示 A 的帮助下，确定性图灵机能够在多项式时间内判定的语言组成 P^A，非确定性图灵机对应的语言类记为 NP^A。如果问题 B 可以通过多项式时间计算和对 A 的多次查询解决，就称 B 可 **Cook 归约（Cook reduction）**到 A，也称多项式时间 Turing 归约。查询可以根据此前的回答调整。例如，判断 UNSAT 时，可以查询同一公式是否属于 SAT，再将答案取反。相比之下，前面的 Karp 归约要求一次实例转换，并保持判定答案相同。

> **定理 10.21** (Baker–Gill–Solovay)**.** 存在语言 A, B，使 $\mathsf{P}^A = \mathsf{NP}^A$，但 $\mathsf{P}^B \neq \mathsf{NP}^B$。

相等的例子可以取 $A = \mathrm{TQBF}$，此时 $\mathsf{P}^A = \mathsf{NP}^A = \mathsf{PSPACE}$。不相等的例子则通过对角线法构造，让非确定性图灵机可以猜出一个字符串并查询它，而每台确定性多项式时间图灵机都因查询不够而至少答错一次。

如果给证明中的确定性和非确定性图灵机增加同一个任意谕示后，证明仍然成立，就称这种证明**可相对化（relativizing）**。假如用可相对化的证明得到了 $\mathsf{P} = \mathsf{NP}$，加入任意谕示后也应当相等，但定理中的 B 却使 $\mathsf{P}^B \neq \mathsf{NP}^B$。反过来，假如用这种证明得到了 $\mathsf{P} \neq \mathsf{NP}$，加入任意谕示后也应当不等，又与 $\mathsf{P}^A = \mathsf{NP}^A$ 矛盾。因此，可相对化的证明既不能证明 $\mathsf{P} = \mathsf{NP}$，也不能证明 $\mathsf{P} \neq \mathsf{NP}$。前面证明时间层次定理时使用的对角线论证可以相对化，因此不能照搬来解决 P 与 NP 是否相等的问题。

10.6.4 困难性假设与近期进展

证明一个问题是 NP 完全的以后，我们仍可能希望知道，它的算法能比枚举快多少。为研究这些更具体的时间界，复杂度理论提出了一些困难性假设，并据此推导其他问题的条件下界。下面介绍两类这样的假设及近期进展。

指数时间假设

含 n 个变量的 3-SAT 可以通过枚举 2^n 个赋值求解。能否让指数从 n 降到比 n 增长更慢的函数？**指数时间假设（Exponential Time Hypothesis，ETH）**认为做不到，即不存在时间为 $2^{o(n)}L^{O(1)}$ 的确定性算法，其中 L 是公式的输入长度。

2026 年 10 月，OpenAI 与 Anthropic 分别公开了证伪 ETH 的算法与证明材料，给出了 3-SAT 的次指数时间算法。Anthropic 还公开了相应的 Lean 形式化证明工程。[4]这些算法突破了 ETH 所预测的指数时间障碍，但所给时间界仍高于多项式，因此尚未解决 P 与 NP 是否相等的问题。

ETH 曾用于证明许多条件下界。例如，假设有一个多项式时间归约，将含 n 个变量的 3-SAT 公式转成问题 Q 的实例，并且新实例的长度 $N = O(n^2)$。如果 Q 有 $2^{o(\sqrt{N})}$ 时间算法，就可以先归约再调用它，用 $2^{o(n)}$ 时间加上归约时间解决原来的 3-SAT 问题。过去，根据 ETH 可以排除这样的 Q 的算法。ETH 被推翻后，归约本身仍然成立，但原来的假设已经无法支持这个下界。

2026 年 11 月的 *ETH is Dead, Long Live SH* 讨论了 Stearns–Hunt 可满足性假设，尝试用它继续研究 3-SAT 的时间下界。论文证明，这一假设等价于要求，对于每个固定的 $0 < \varepsilon < 1$，3-SAT 都没有 $O(2^{n^{1-\varepsilon}})L^{O(1)}$ 时间算法。ETH 排除所有次指数时间算法，而这一假设只排除把指数从 n 降到 $n^{1-\varepsilon}$ 的算法，例如 $2^{\sqrt{n}}L^{O(1)}$ 时间算法。因此，它比 ETH 弱，可以与某些次指数时间算法并存。在这一假设下，许多归约仍能给出较弱的条件下界。

3SUM 与全源最短路

即使已知一个问题属于 P，我们仍希望找到更快的算法。例如，3SUM 要判断给定的 n 个数中，是否有三个不同位置的数之和为 0。将这些数排序后，逐个固定一个数，再用双指针寻找另外两个数，就能在 $O(n^2)$ 时间内解决。第八讲的 Floyd–Warshall 算法则用 $O(n^3)$ 时间求出全源最短路。长期以来，研究者猜测这两个指数无法降低一个固定的正常数，这分别称为 3SUM 假设和 APSP 假设。

Alman 与 Vassilevska Williams 于 2026 年 10 月公开的预印本突破了这两个时间界。在字 RAM 模型下，对绝对值由 n 的多项式界定的整数，论文给出了 $O(n^{1.9992})$ 时间的确定性 3SUM 算法。对于边权也满足这一整数范围限制、且不含负环的有向图，论文给出的全源最短路算法只需 $O(n^{2.9995})$ 时间。虽然指数的改进很小，却已经足以推翻相应的困难性假设。

[4]见参考资料中的 *The Exponential Time Hypothesis is False* 及 *formal-math* 的 eth 项目。后者还给出了固定子句宽度下计算满足赋值个数的时间界。

10.7 习题

习题 10.1 (拼接证书)**.** 设语言 L_1, L_2 都属于 NP，证明 L_1L_2 也属于 NP。再设 $L \in \mathsf{NP}$，证明 $L^* \in \mathsf{NP}$。

习题 10.2 (如果 P 等于 NP)**.** 假设 $\mathsf{P} = \mathsf{NP}$。证明：对任意 $A \in \mathsf{NP}$，只要 A 至少包含一个字符串且至少有一个字符串不属于 A，那么 A 就是 NP 完全的。为什么需要后面这两个条件？据此判断以下问题在这一假设下是否为 NP 完全问题：给定一个无权无向图、两个顶点 s, t 和非负整数 k，是否存在一条从 s 到 t、边数不超过 k 的路径？

习题 10.3 (永真式)**.** 若布尔公式在每一种赋值下都为真，就称它为永真式。

1. 证明：判断一个一般布尔公式是否为永真式，是 co-NP 完全问题。
2. 如果输入已经是 CNF 公式，能否在多项式时间内判断它是不是永真式？说明算法及理由。

习题 10.4 (0–1 整数规划)**.** 给定整数矩阵 $A \in \mathbb{Z}^{m\times n}$ 和整数向量 $b \in \mathbb{Z}^m$，所有整数均用二进制表示。我们希望为每个变量 x_i 选择 0 或 1，使 $Ax \leqslant b$，即每一行对应的不等式都成立。证明判断这样的 x 是否存在是 NP 完全问题。

习题 10.5 (度数受限的生成树)**.** 给定一个连通简单无向图 G，是否能选出一棵生成树，使树中每个顶点的度数都不超过 2？证明这个问题是 NP 完全的，可以从 Hamilton 回路问题出发。

习题 10.6 (分成总和相等的两组)**.** 考虑如下问题：给定正整数 $a_1, \ldots, a_n$，能否将它们分成两组，使两组的总和相等？请从子集和问题出发，证明这个问题是 NP 完全的。

习题 10.7 (背包的困难性)**.** 给定 n 件物品，第 i 件的重量为 s_i、价值为 v_i，背包容量为 C，另给目标价值 V。这些数均为用二进制表示的非负整数，每件物品最多选一次。

1. 从子集和问题出发，证明判断能否选出总重量不超过 C、总价值至少为 V 的物品，是 NP 完全问题。由此说明求最大价值的优化问题是 NP 困难的。
2. 若每件物品的重量都不超过 n^2，而价值和容量的大小不受这一限制，能否在输入长度的多项式时间内精确求出最大价值？请说明算法及其时间复杂度。

10.8 参考答案

习题 *10.1.* 提示：给出带有切分位置的证书，然后对每一段字符串和相应的证书运行验证算法，都接受才能接受。 □

习题 *10.2.* 固定两个字符串 $y \in A$、$z \notin A$。对任意 $L \in \mathsf{NP}$，由 $\mathsf{P} = \mathsf{NP}$，可以先在多项式时间内判断 $x \in L$ 是否成立，再在成立时输出 y，不成立时输出 z。这就构造了 $L \leqslant_P A$。结合 $A \in \mathsf{NP}$，可知 A 是 NP 完全的。这两个条件保证归约能分别输出肯定实例和否定实例。若 $A = \varnothing$，无论输出什么，都得到否定答案，无法对应原问题的肯定实例。若 $A = \Sigma^*$，则所有输出都给出肯定答案，无法对应原问题的否定实例。因此，这两个语言都不能是 NP 困难的。

题中的路径问题可以用 BFS 求出最少边数，再与 k 比较，所以属于 P。它同时有肯定实例和否定实例，例如两个顶点之间有边或无边，均取 $k = 1$。由第一问，在 $\mathsf{P} = \mathsf{NP}$ 的假设下，它也是 NP 完全的。 □

习题 *10.3.* 第一问，公式不是永真式时，一个使它为假的赋值就是证书，因此 TAUT $\in$ co-NP。由 SAT 的 NP 完全性，UNSAT 是 co-NP 完全问题。又因为 F 不可满足当且仅当 $\neg F$ 是永真式，取否定就给出了 UNSAT $\leqslant_P$ TAUT，故 TAUT 是 co-NP 完全问题。

第二问，CNF 要在每种赋值下都为真，每个子句都必须在每种赋值下为真，故只需逐个判断子句是不是永真式，而子句由析取连接，很容易找到多项式时间算法判断。 □

习题 *10.4.* 证书的长度显然是多项式的，给出证书的验证也显然可以在多项式时间内完成，故问题属于 NP。为了证明 NP 困难性，我们把 3-SAT 归约到这个整数规划问题，使满足赋值恰好对应可行的整数解。对每个布尔变量设置一个 0–1 变量，将正文字 x_i 替换为 x_i，负文字 $\neg x_i$ 替换为 $1 - x_i$，并要求每个子句中这些表达式之和至少为 1。例如 $(x_1 \vee \neg x_2 \vee x_3) \mapsto x_1 + (1 - x_2) + x_3 \geqslant 1$，整理后就是 $-x_1 + x_2 - x_3 \leqslant 0$。每个子句产生一行约束，而且该约束成立当且仅当子句为真。因此原公式可满足当且仅当整数规划有解。构造及输出长度均为多项式，故问题是 NP 完全的。 □

习题 *10.5.* 一棵至少有两个顶点且所有顶点度数都不超过 2 的树必是一条路径。因此题目等价于判断 G 是否有 Hamilton 路径。给出路径后，可以在多项式时间内检查顶点及边，所以问题属于 NP。

下面将 Hamilton 回路归约到本题。对至少有三个顶点的连通简单无向图，任选一个顶点 v，把它拆成 v'、v''，让这两个顶点分别与 v 原来的所有邻居相连。再增加两个顶点 s, t，分别只与 v'、v'' 相连。若原图有 Hamilton 回路，在 v 处断开回路，就能在新图中得到从 s 经 v'、原图其余顶点、v'' 到 t 的 Hamilton 路径。反过来，新图中的 s, t 度数都为 1，任何 Hamilton 路径都必须以它们为端点。删去这两个端点，再把 v'、v'' 合为 v，便得到原图的 Hamilton 回路。新图仍连通，构造只增加常数个顶点和 $O(|V|)$ 条边。原图不连通或不足三个顶点时输出没有符合要求的生成树。因此归约对所有输入成立，本题是 NP 完全的。 □

习题 *10.6.* 给出分组后，分别求和即可在多项式时间内验证，所以问题属于 NP。对于困难性，考虑子集和实例 $a_1, \ldots, a_n$ 和目标 t，记 $S = \sum_i a_i$。当 $0 \leqslant t \leqslant S$ 时，在原来的数中加入 $b = S + 1$，$c = 2t + 1$。全部数的总和为 $2(S + t + 1)$，所以每组必须恰好为 $S + t + 1$。因为 $b + c$ 比这个值大 $t + 1$，所以 b，c 必须分到不同组。在含 b 的那一组中，其余数的和必须为 t，因而能从原输入中找到总和为 t 的子集。反过来，如果原输入有一个总和为 t 的子集，就将它与 b 放在同一组，剩余数与 c 放在另一组。两组的和都是 $S + t + 1$，于是得到所需的等和分组。若 $t < 0$ 或 $t > S$，输出无法等分即可。新增整数的二进制位数为 $O(\log S)$，构造是多项式时间，故问题是 NP 完全的。 □

习题 *10.7.* 第一问，给出选中物品的下标后，求出总重量和总价值即可验证，所以背包判定属于 NP。对子集和实例 $(a_1, \ldots, a_n, B)$，令 $s_i = v_i = a_i$，$C = V = B$。由于重量与价值相同，总重量不超过 B 且总价值至少为 B，恰好要求所选数字的和等于 B。构造只需复制这些数字，因而是多项式时间归约，背包判定是 NP 完全的。若能求出最大价值，再与 V 比较就能回答判定问题，所以优化问题是 NP 困难的。

第二问，可以。所有物品的总重量至多为 $n \cdot n^2 = n^3$。如果容量 C 比总重量还大，将它减到总重量也不会影响任何选取方案。因此，可以把容量改为 $C' = \min\{C, \sum_i s_i\}$，此时 $C' \leqslant n^3$。再使用第八讲按容量填表的动态规划，至多需要 $O(n^4)$ 的时间。 □

致谢

在此我需要非常感谢刘泓健同学为本章讲义的绝大部分内容提供的基本框架。我们真诚地希望这几章的讲义的作用不仅仅是让读者理解课程需要掌握的内容,更重要的是在这片理论计算机科学尚不繁荣的土地上——毕竟在这里笔者感兴趣的算法博弈论和机器学习理论都能算得上很理论的——种下一颗种子,让读者去探索更多相关的知识。也由衷感谢 Prof. Arora 和 Prof. Barak 的 *Computational Complexity: A Modern Approach* 一书，本章的许多内容都参考了这本书。

作为世界一流大学的世界一流专业，在计算机科学与技术学院没有复杂度理论的专题课程是非常令人惊讶的。这个领域本应该是计算机科学的重要组成部分，其中也反映了许多计算机科学的基本思想和方法，诞生了许多图灵奖得主（包括 Cook 和 Karp 等我们在本节中见到的名字）。但是，我们遗憾地看到，计算机科学与技术的学生对科学一无所知，对技术也知之甚少。我们希望，这份讲义（尤其是其中的对角线方法）能够让大家体会到计算机科学本身的精妙之处，并且能够更多地了解相关的内容，比方说可计算性理论、数理逻辑等等。在这一讲中，我们尚未能明确给出数理逻辑和复杂度理论之间的直接关联，虽然我们通过几个例子将其展现了一部分。有兴趣的读者可以寻找一些多项式层级、描述复杂度等关键词的文献，它更明显地表达了复杂度和数理逻辑之间的联系，对于自动定理证明、理论求解等领域也有诸多应用。此外，复杂度理论尚有一种更加现代化、形式化的表述方式，即所谓复杂度测度。算法信息论中的 Kolmogorov 复杂度用生成一个对象的最短程序长度衡量其描述复杂度。此外，谈到数理逻辑，值得一提的就是与之并称的类型论和范畴论。例如综合复杂度理论（synthetic complexity theory）尝试从抽象的计算与资源公理出发研究复杂度，Asperti 的间隙定理形式化工作展示了这一方向的一种做法。但是，这种未定的形态更适合那些有志于计算机科学的研究者，毕竟，这种仿佛来自虚空的和谐之美，是我们不变的追求。

本章介绍的近期进展，也让我们开始看到前沿大语言模型的理论研究能力，就像是看到“原子弹爆炸”一样。大语言模型强大数学推理能力的涌现，加速了人类对理论探索的进程，但同时也引发了数学界的焦虑与不满。在笔者写下这些内容的时候，我们仍无法判断大语言模型的能力边界，我们仍在探寻大语言模型时代理论研究者的未来该何去何从。当然，无论未来的研究方式如何改变，我们仍希望，理解一个深刻思想时的喜悦，能够继续吸引人们走进这片天地。

10.9 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十讲。
2. Cormen, Leiserson, Rivest, Stein，*Introduction to Algorithms*，第 3 版，第 22.5 节及第 34 章。介绍强连通分量、输入编码、NP 完全性及团、顶点覆盖、子集和的归约。相关练习见 34.4-4、34.4-7。
3. Kleinberg, Tardos，*Algorithm Design*，第 8–9 章。介绍多项式归约、计算困难性与空间复杂度。
4. Arora, Barak，*Computational Complexity: A Modern Approach*，第 1 章附录及第 2.3、3.1–3.5、4.3、5.2、8.3 节。包括通用模拟、Cook–Levin 定理、复杂度层次与空间复杂度。https://theory.cs.princeton.edu/complexity/。
5. Welch, P. D.，*A First Course on Logic*，1996，第 7 章。对角线引理与 Gödel–Rosser 定理分别见 7.25 和 7.34。https://people.maths.bris.ac.uk/~mapdw/pdw-logic-revised.pdf。
6. Asperti, A.，*A Formal Proof of Borodin–Trakhtenbrot's Gap Theorem*，CPP 2013 报告。介绍间隙定理

的构造及其形式化证明。https://www.cs.cornell.edu/courses/cs6110/2015sp/docs/Borodin-Gap-theorem.pdf。

7. Sedgewick, Wayne, *Algorithms*, 第 4 版, 4.2 节配套资料: 强连通分量与 2-SAT。https://algs4.cs.princeton.edu/42digraph/。
8. Clay Mathematics Institute，P versus NP 问题介绍及未解问题列表。https://www.claymath.org/millennium-problems/。
9. Williams, R. R.，*Simulating Time With Square-Root Space*，2025。研究如何用更少的空间模拟给定时间内的计算。arXiv:2502.17779。
10. Alman, J., Vassilevska Williams, V., *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*，2026 年 10 月预印本。arXiv:2610.06783。
11. OpenAI，*The Exponential Time Hypothesis is False*，2026 年 10 月 25 日预印本。论文与发布说明。
12. Anthropic，*formal-math*，2026。eth 与 3sum-apsp 项目包含论文、Lean 证明及构建说明。公开证明工程。
13. MIT Hardness Group 等，*ETH is Dead, Long Live SH*，2026 年 11 月 2 日预印本。讨论 ETH 被推翻后仍可使用的困难性假设。arXiv:2611.02650。